Introduction to Deep Learning

Caltech EE/CS/CNS 148a - Winter 2026
P. Perona

Lecture 2

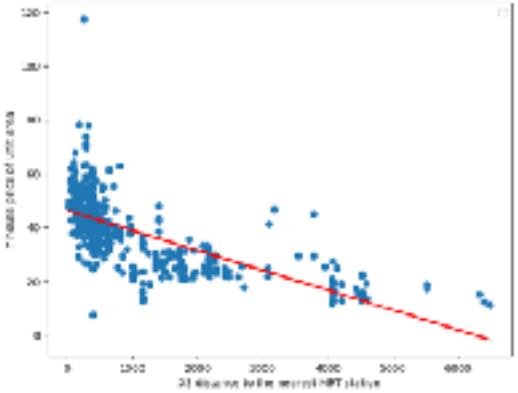
Non-linearities, loss functions, back propagation, gradient descent

Recap

Task categories

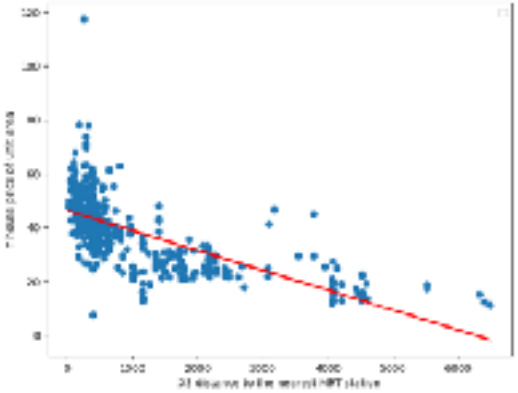
Regression

$$f : X \in \mathbb{R}^m \rightarrow Y \in \mathbb{R}^n$$



Regression

$$f : X \in \mathbb{R}^m \rightarrow Y \in \mathbb{R}^n$$

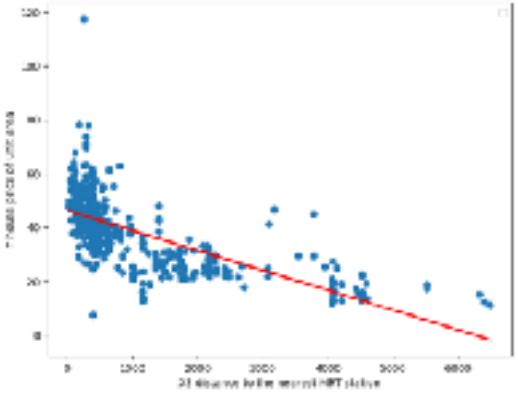


Classification

$$f : X \in \mathbb{R}^n \rightarrow Y \in \{C_1, C_2, \ldots, C_m\}$$

Regression

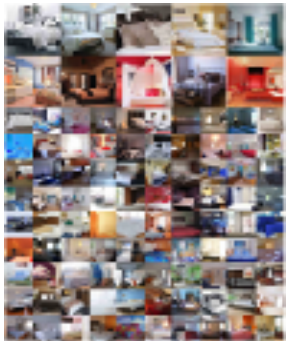
$$f : X \in \mathbb{R}^m \rightarrow Y \in \mathbb{R}^n$$



Classification

$$f : X \in \mathbb{R}^n \rightarrow Y \in \{C_1, C_2, \ldots, C_m\}$$

Probability density estimation



Key elements of learning

Data

Input-output pairs that fit the task.

$$\mathcal{D} = \{x_i, y_i\}$$

X1 transaction date	X2 house age	X3 distance to the nearest MRT station	X4 number of convenience stores	X5 latitude	X6 longitude	Y house price of unit area
2012.917	32.0	84.87882	10	24.98298	121.54024	37.9
2012.917	19.5	306.59470	9	24.98034	121.53951	42.2
2013.583	13.3	561.98450	5	24.98746	121.54391	47.3
2013.500	13.3	561.98450	5	24.98746	121.54391	54.8
2012.833	5.0	390.56840	5	24.97937	121.54245	43.1

Input

Output

Class of functions

Which functions do I want to consider as candidates to solve the task?

$$\boxed{f \in \mathcal{F}}$$

$$f : X \longrightarrow Y$$

Input Output

Selection / learning criterion

How do I select the best function to solve the task, using the data I have?

$$D = \{x_i, y_i\}$$

$$\boxed{f = \arg\min_{f \in \mathcal{F}} \mathcal{L}(f, D)}$$

Selection / learning criterion

Training set

$$D = \{x_i, y_i\}$$

How do I select the best function to solve the task, using the data I have?

$$\boxed{f = \arg\min_{f \in \mathcal{F}} \mathcal{L}(f, D)}$$

Selection / learning criterion

Training set

$$D = \{x_i, y_i\}$$

Loss function

How do I select the best function to solve the task, using the data I have?

$$\boxed{f = \arg\min_{f \in \mathcal{F}} \mathcal{L}(f, D)}$$

Evaluation

How do I decide which one of various trained models I select for future use?

$$\mathcal{T} = \{x_i, y_i\}$$

$$\mathcal{M} : (\bar{f}, \mathcal{T}) \longrightarrow \mathbb{R}$$

Evaluation

Test set

$$\mathcal{T} = \{x_i, y_i\}$$

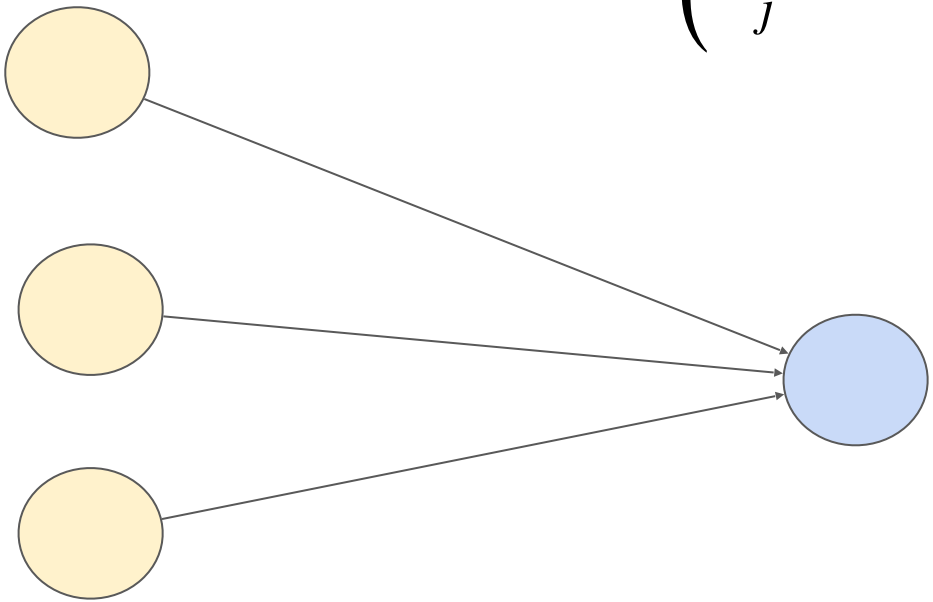
How do I decide which one of various trained models I select for future use?

$$\mathcal{M} : (\bar{f}, \mathcal{T}) \longrightarrow \mathbb{R}$$

Class of functions: Neural Networks

Unit

$$x_i^{(n)} = \sigma\left(\sum_j w_{ij}^{(n)} x_j^{(n-1)} + b_i^{(n)} \right)$$

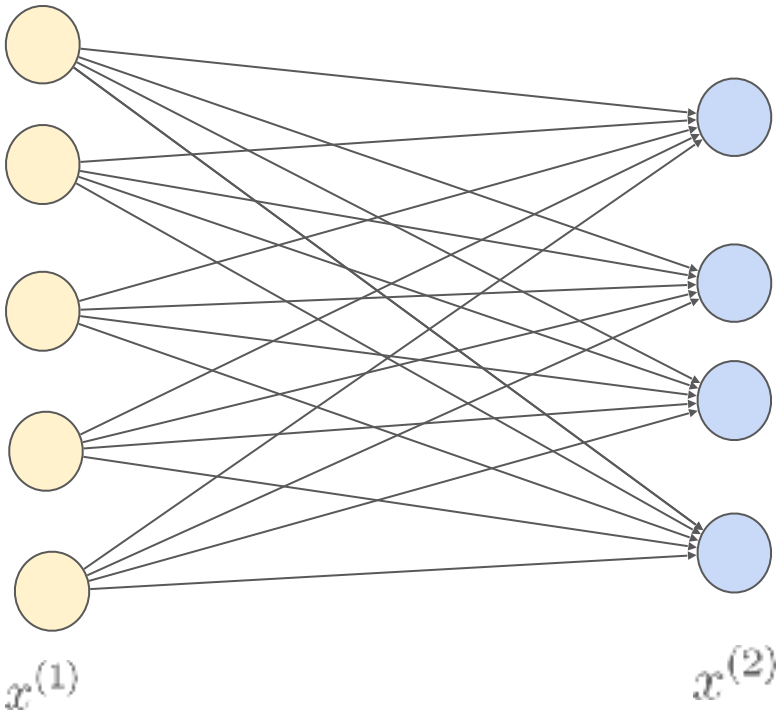


Real input variables

Real (or discrete) output variable

Real weights

Layers

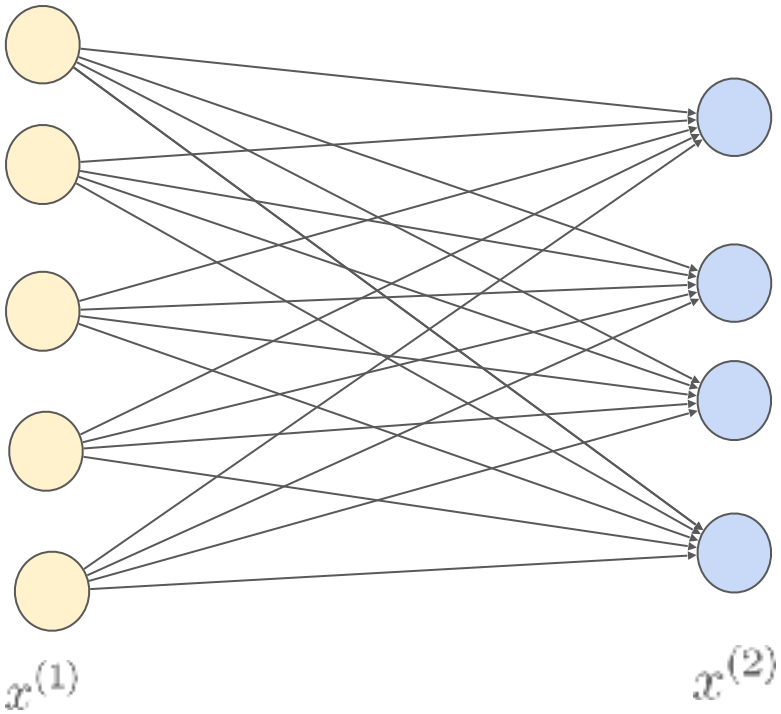


Layer

Layer

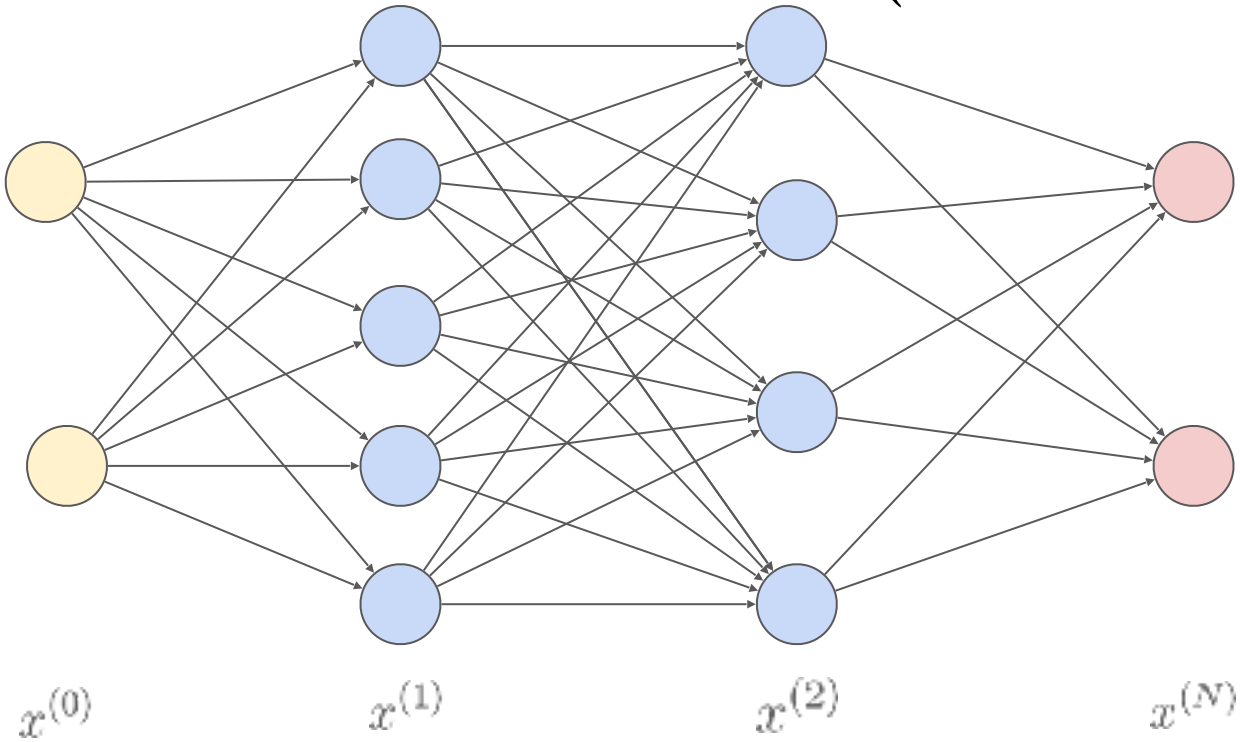
$$x_i^{(n)} = \sigma\left(\sum_j w_{ij}^{(n)} x_j^{(n-1)} + b_i^{(n)} \right)$$

Index within layer



Multi-layer

$$x_i^{(n)} = \sigma\left(\sum_j w_{ij}^{(n)} x_j^{(n-1)} + b_i^{(n)} \right)$$



Just linear functions?

$$x_i^{(n)} = \sum_j w_{ij}^{(n)} x_j^{(n-1)} + b_j^{(n-1)}$$

The good

The bad

Just linear functions?

$$x_i^{(n)} = \sigma\left(\sum_j w_{ij}^{(n)} x_j^{(n-1)} + b_i^{(n)} \right)$$

The good

Simple to understand and represent

Robust to outliers

Easy to learn

The bad

Not sufficiently expressive

Makes depth useless

Introduce nonlinearity

σ

$$x_i^{(n)} = \sigma\left(\sum_j w_{ij}^{(n)} x_j^{(n-1)} + b_i^{(n)} \right)$$

Goal: make neural networks “universal approximators”

Introduce nonlinearity

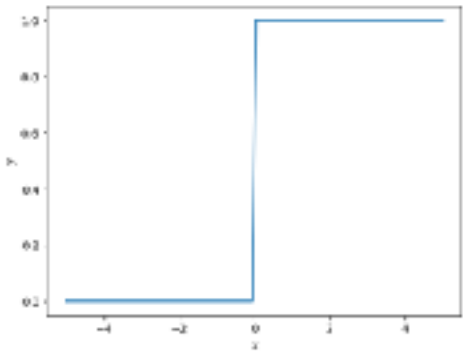
σ

$$x_i^{(n)} = \sigma\left(\sum_j w_{ij}^{(n)} x_j^{(n-1)} + b_i^{(n)} \right)$$

Goal: make neural networks “universal approximators”

Idea # 1

$$\sigma(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$$



Introduce nonlinearity

σ

$$x_i^{(n)} = \sigma\left(\sum_j w_{ij}^{(n)} x_j^{(n-1)} + b_i^{(n)} \right)$$

Goal: make neural networks “universal approximators”

Idea # 1: Step Function

$$\sigma(x) = \begin{cases} 1, & x \geq \theta \\ 0, & x < \theta \end{cases}$$

Introduce nonlinearity

σ

$$x_i^{(n)} = \sigma\left(\sum_j w_{ij}^{(n)} x_j^{(n-1)} + b_i^{(n)} \right)$$

Goal: make neural networks “universal approximators”

Idea # 1: Step Function

$$\sigma(x) = \begin{cases} 1, & x \geq \theta \\ 0, & x < \theta \end{cases}$$

It’s good at approximating functions!

Introduce nonlinearity

σ

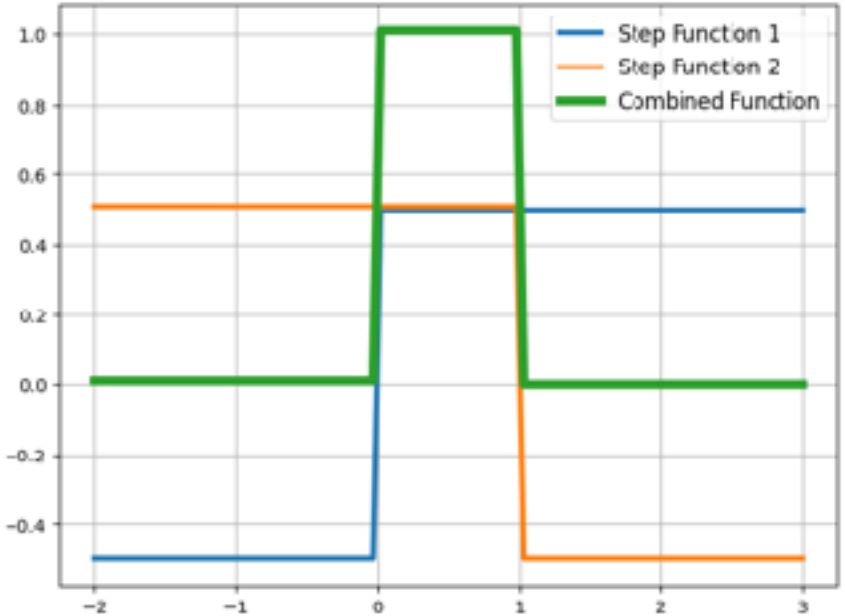
$$x_i^{(n)} = \sigma\left(\sum_j w_{ij}^{(n)} x_j^{(n-1)} + b_i^{(n)} \right)$$

Goal: make neural networks “universal approximators”

Idea # 1: Step Function

$$\sigma(x) = \begin{cases} 1, & x \geq \theta \\ 0, & x < \theta \end{cases}$$

It’s good at approximating functions!



Introduce nonlinearity

σ

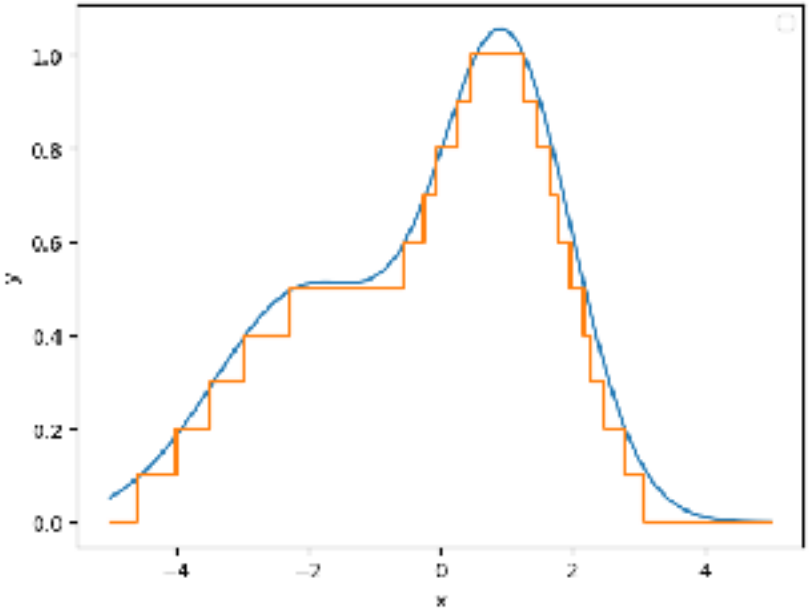
$$x_i^{(n)} = \sigma\left(\sum_j w_{ij}^{(n)} x_j^{(n-1)} + b_i^{(n)} \right)$$

Goal: make neural networks “universal approximators”

Idea # 1: Step Function

$$\sigma(x) = \begin{cases} 1, & x \geq \theta \\ 0, & x < \theta \end{cases}$$

It’s good at approximating functions!



Introduce nonlinearity σ

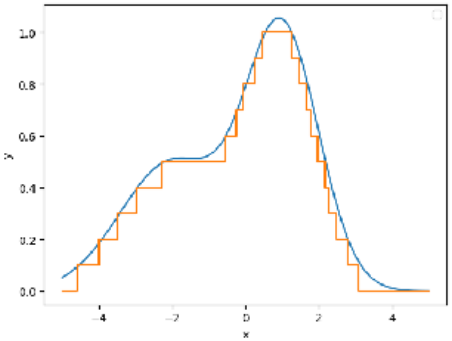
$$x_i^{(n)} = \sigma\left(\sum_j w_{ij}^{(n)} x_j^{(n-1)} + b_i^{(n)}\right)$$

$$\boxed{w = w + \eta \frac{d}{dW}\mathcal{L}(x, w)}$$

Goal: make neural networks “universal approximators”
Goal: **make it easy to learn the parameters**

Idea # 1: Step Function

$$\sigma(x) = \begin{cases} 1, & x \geq \theta \\ 0, & x < \theta \end{cases}$$



It’s good at approximating functions!

Introduce nonlinearity σ

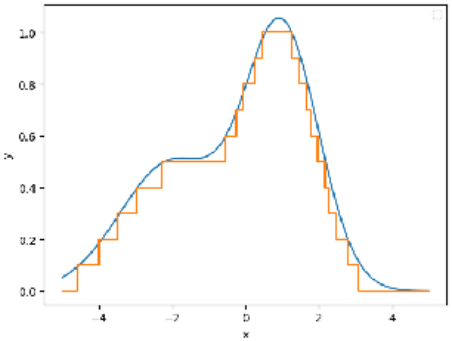
$$x_i^{(n)} = \sigma\left(\sum_j w_{ij}^{(n)} x_j^{(n-1)} + b_j^{(n-1)}\right)$$

$$w = w + \eta \frac{d}{dW} \mathcal{L}(x, w)$$

Goal: make neural networks “universal approximators”
Goal: make it easy to learn the parameters

Idea # 1: Step Function

$$\sigma(x) = \begin{cases} 1, & x \geq \theta \\ 0, & x < \theta \end{cases}$$



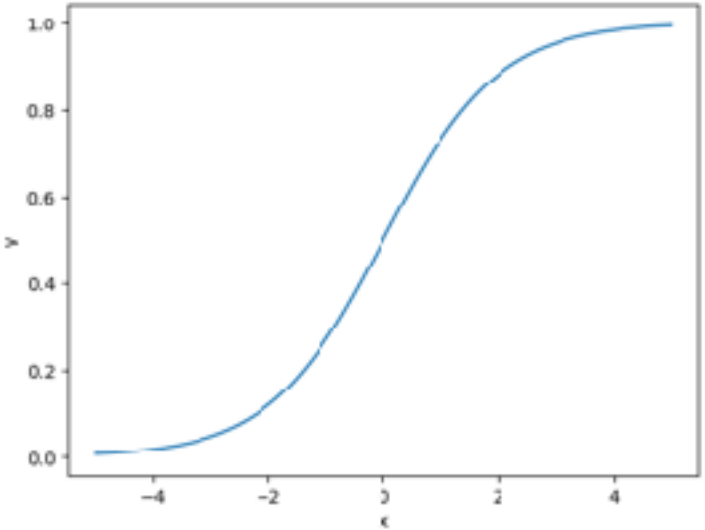
It’s good at approximating functions!

It’s not differentiable.

Logistic function

$$x_i^{(n)} = \sigma\left(\sum_j w_{ij}^{(n)} x_j^{(n-1)} + b_i^{(n)} \right)$$

$$\sigma(x) = \frac{1}{1 + e^{-kx}}$$



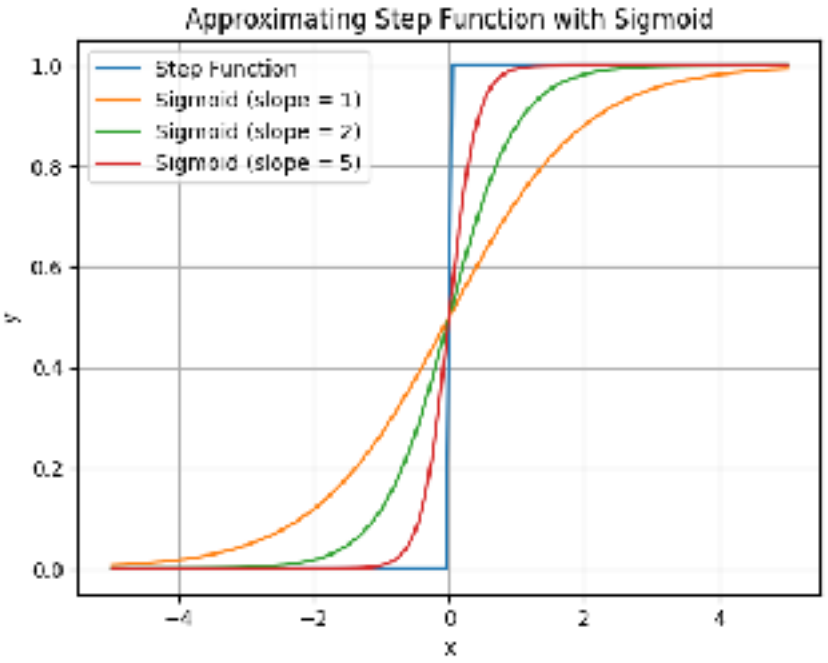
Can approximate functions as well

It is differentiable!

Logistic function

$$x_i^{(n)} = \sigma\left(\sum_j w_{ij}^{(n)} x_j^{(n-1)} + b_i^{(n)} \right)$$

$$\sigma(x) = \frac{1}{1 + e^{-kx}}$$



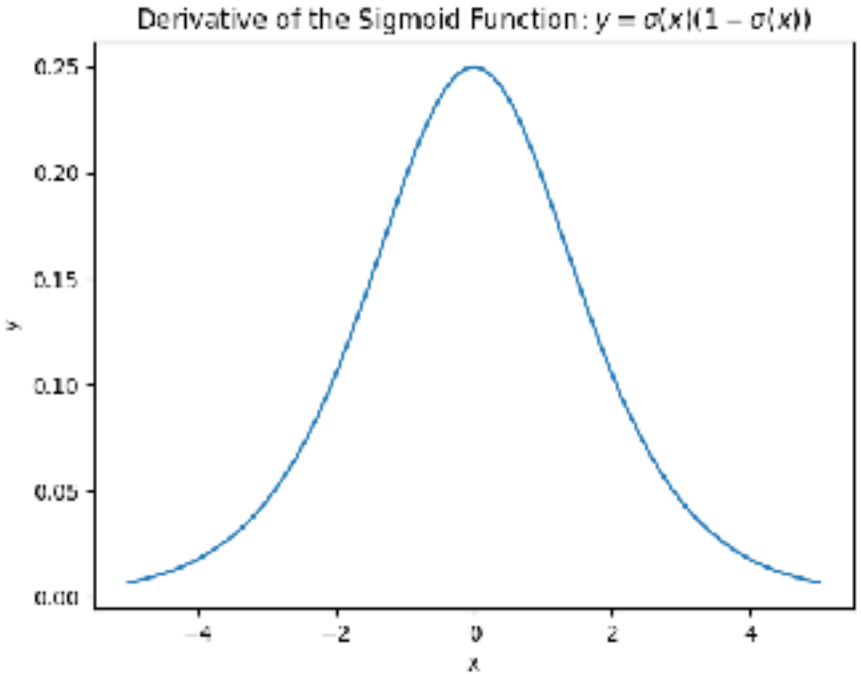
Can approximate functions as well

It is differentiable!

Logistic function

$$x_i^{(n)} = \sigma\left(\sum_j w_{ij}^{(n)} x_j^{(n-1)} + b_i^{(n)} \right)$$

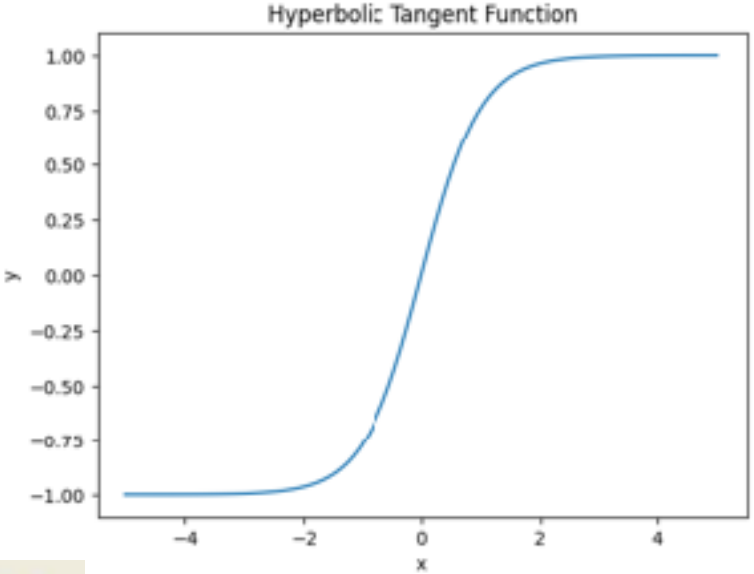
$$\sigma(x) = \frac{1}{1 + e^{-kx}}$$

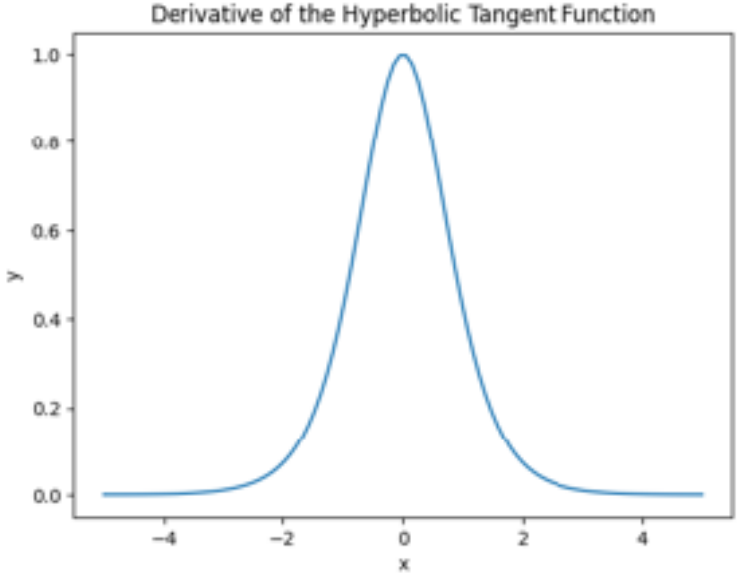


The gradient vanishes fast

Hyperbolic tangent

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

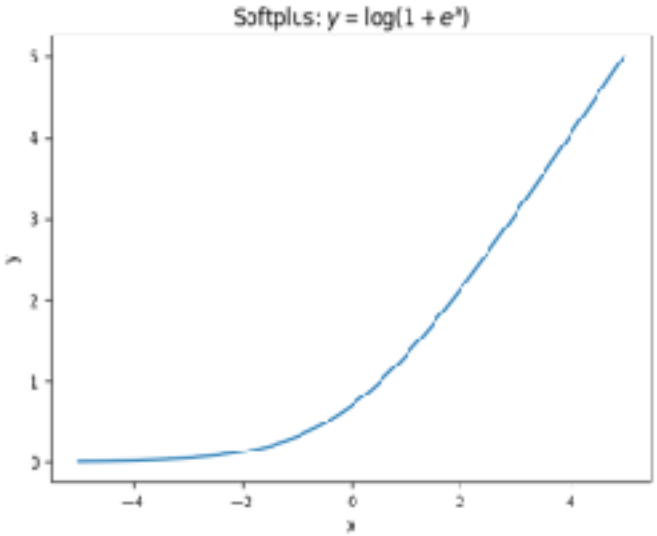


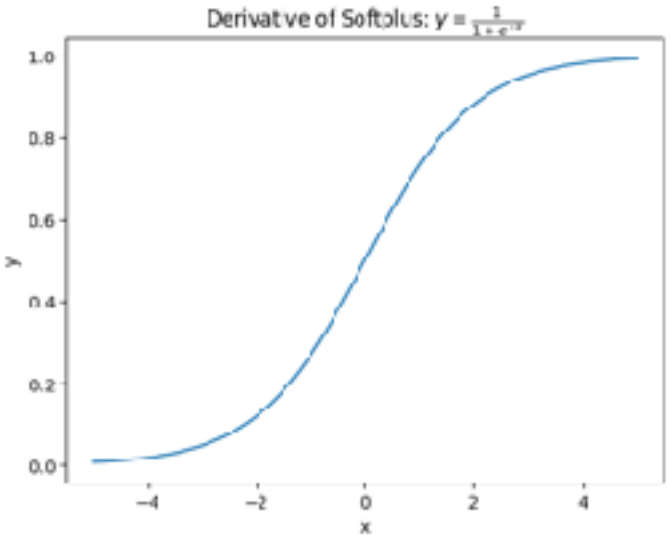


Similar to logistic - belongs to the same class of sigmoid functions
Behaves better in learning as it centered in 0

Softplus

$$y = \frac{1}{1 + e^{-x}}$$

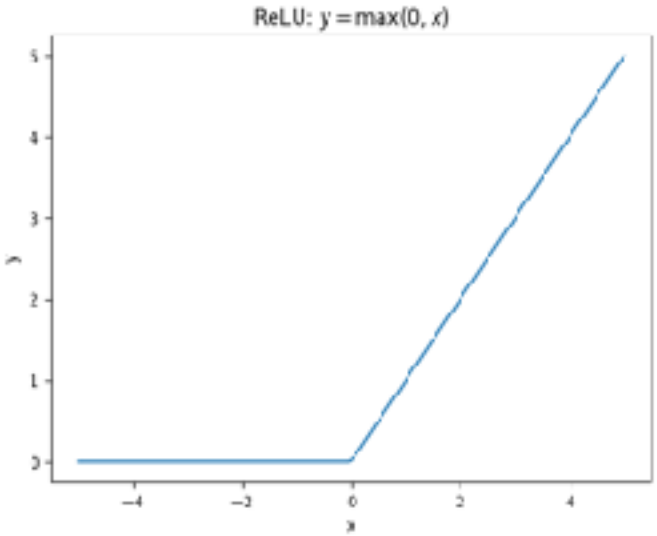


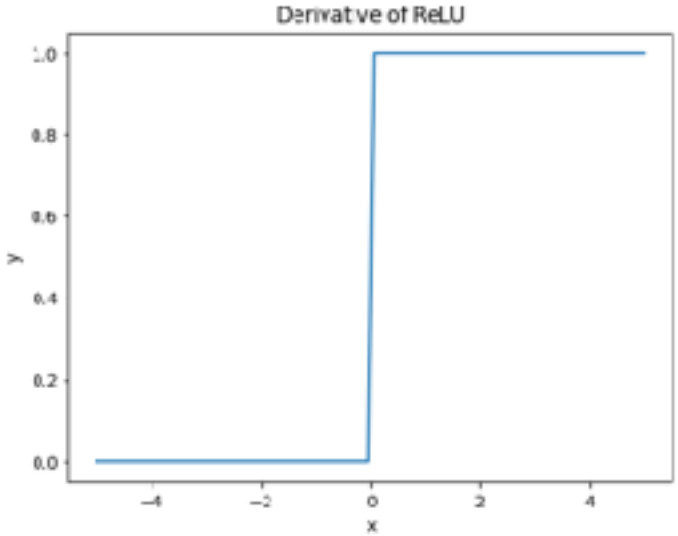


Getting there… It's still good for approximation and the gradient is looking better!

Make it simpler: ReLu

$$y = max\{0, x\}$$



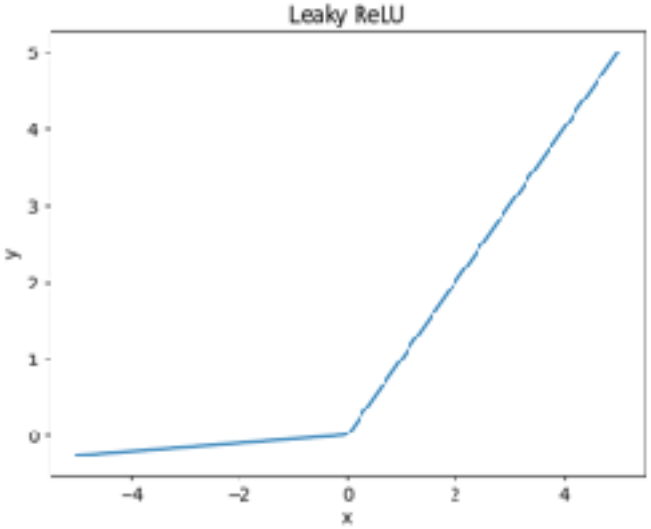


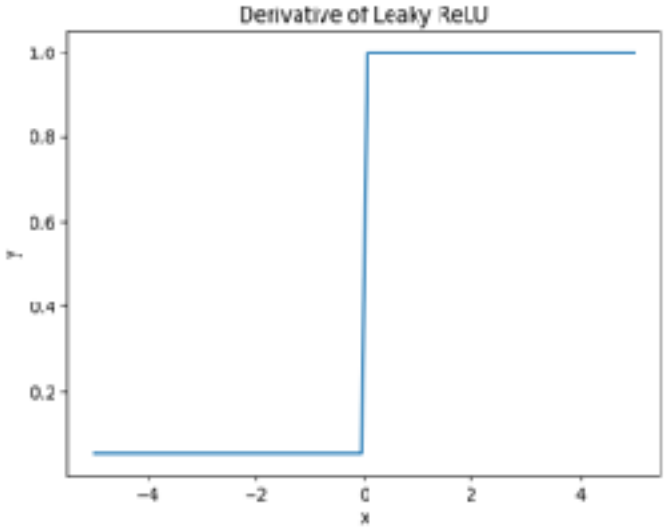
Well, it's as good as Softplus but much simpler!
We still may have the vanishing gradient problem

Leaky ReLu

$$y = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}$$

$$y = \begin{cases} 1, & x > 0 \\ \alpha, & x \leq 0 \end{cases}$$





This helps with gradient in the negative range

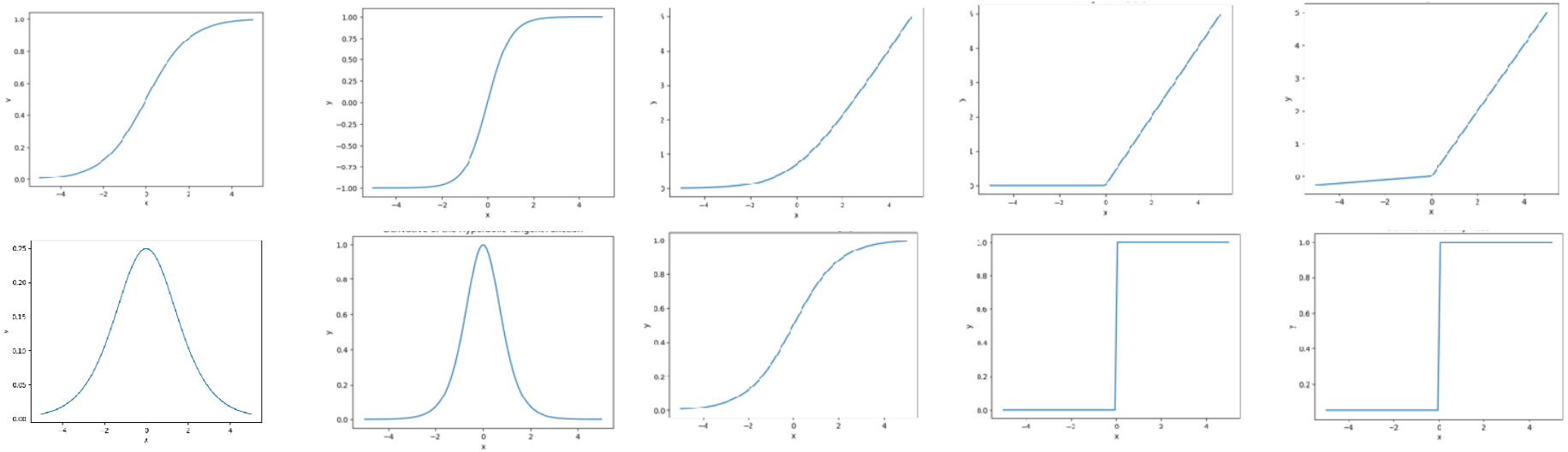
Nonlinearities or activations: recap

$$x_i^{(n)} = \sigma\left(\sum_j w_{ij}^{(n)} x_j^{(n-1)} + b_i^{(n)}\right)$$

σ

Goal: make neural networks “universal approximators”
Goal: make it easy to learn the parameters

$$\sigma(x) = \frac{1}{1+e^{-kx}} \qquad \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \qquad y = \frac{1}{1+e^{-x}} \qquad y = max\{0, x\} \qquad y = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}$$

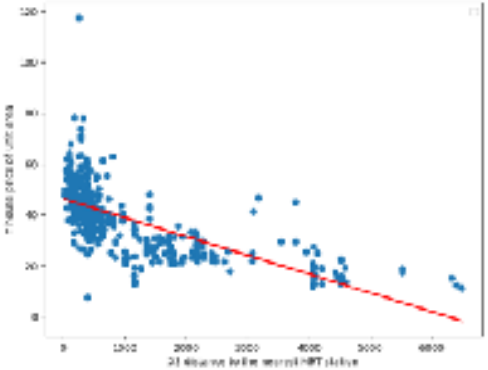


Selection criterion:
the last layer and the loss function

Our tasks

Regression

$$f : X \in \mathbb{R}^m \rightarrow Y \in \mathbb{R}^n$$

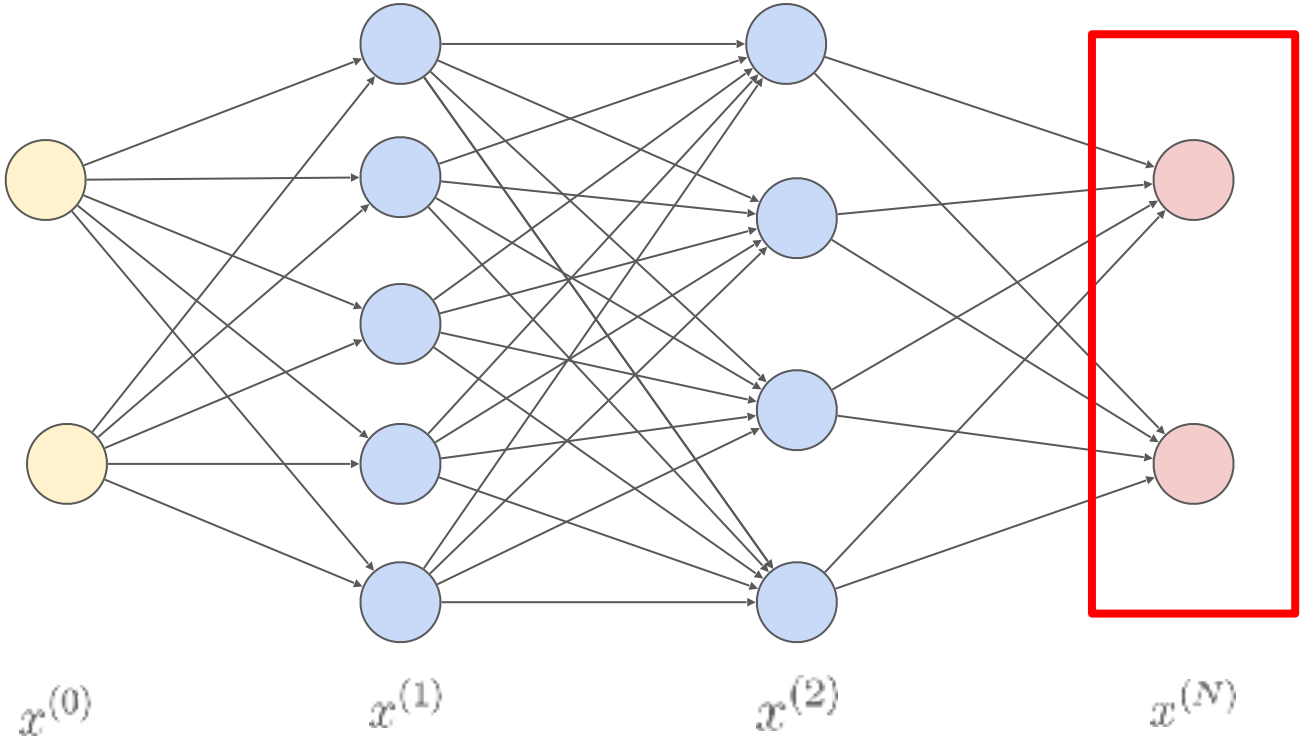


Classification

$$f : X \in \mathbb{R}^n \rightarrow Y \in \{C_1, C_2, \ldots, C_m\}$$

The last layer

$$x_i^{(n)} = \sigma\left(\sum_j w_{ij}^{(n)} x_j^{(n-1)} + b_i^{(n)} \right)$$



Regression: the loss function

$$f(x, w) \in \mathscr{F}$$

$$f(x, w) \in \mathscr{F}$$

$$\mathscr{L}(w) = \int (f(x, w) - y)^2 dx$$

$$f(x, w) \in \mathscr{F}$$

$$\mathscr{L}(w) = \int (f(x, w) - y)^2 dx$$

$$\text{Risk}(f) = \mathbb{E}_{(x,y)}\left[\mathscr{L}(f(x), y)\right]$$

$$f(x, w) \in \mathscr{F}$$

$$\mathscr{L}(w) = \int (f(x, w) - y)^2 dx$$

$$\text{Risk}(f) = \mathbb{E}_{(x,y)}\left[\mathscr{L}(f(x), y)\right]$$

$$w^* = \arg\min_{f \in \mathscr{F}} \text{Risk}(f)$$

$$f(x, w) \in \mathscr{F}$$

$$\mathscr{L}(w) = \int (f(x, w) - y)^2 dx$$

$$\text{Risk}(f) = \mathbb{E}_{(x,y)}\left[\mathscr{L}(f(x), y)\right]$$

$$w^* = \arg\min_{f \in \mathscr{F}} \text{Risk}(f)$$

$$\mathscr{D} = \{(x^{(i)}, y^{(i)})\}_{i=0}^{K}, \quad x \in \mathbb{R}^n, y \in \mathbb{R}$$

$$f(x, w) \in \mathscr{F}$$

$$\mathscr{L}(w) = \int (f(x, w) - y)^2 dx$$

$$\text{Risk}(f) = \mathbb{E}_{(x,y)}\left[\mathscr{L}(f(x), y)\right]$$

$$w^* = \arg\min_{f \in \mathscr{F}} \text{Risk}(f)$$

$$\mathscr{D} = \{(x^{(i)}, y^{(i)})\}_{i=0}^{K}, \quad x \in \mathbb{R}^n, y \in \mathbb{R}$$

$$\mathscr{L}'(w) = \frac{1}{N} \sum_{i=1}^{N} \left(y^{(i)} - f(x^{(i)}, w)\right)^2$$

$$f(x, w) \in \mathscr{F}$$

$$\mathscr{L}(w) = \int (f(x, w) - y)^2 dx$$

$$\text{Risk}(f) = \mathbb{E}_{(x,y)}\left[\mathscr{L}(f(x), y)\right]$$

What is the difference, apart from discrete vs continuous?

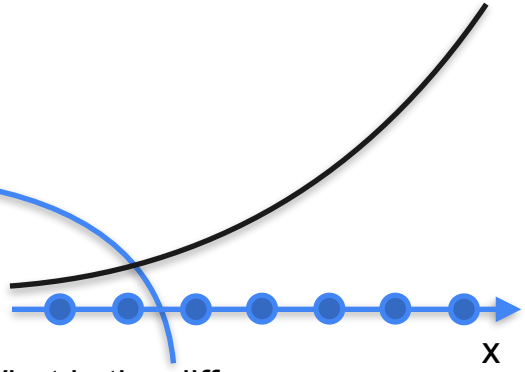
$$w^* = \arg\min_{f \in \mathscr{F}} \text{Risk}(f)$$

$$\mathscr{D} = \{(x^{(i)}, y^{(i)})\}_{i=0}^{K}, \quad x \in \mathbb{R}^n, y \in \mathbb{R}$$

$$\mathscr{L}'(w) = \frac{1}{N} \sum_{i=1}^{N} \left(y^{(i)} - f(x^{(i)}, w)\right)^2$$

$$f(x, w) \in \mathscr{F}$$

$$\mathscr{L}(w) = \int (f(x, w) - y)^2 dx$$



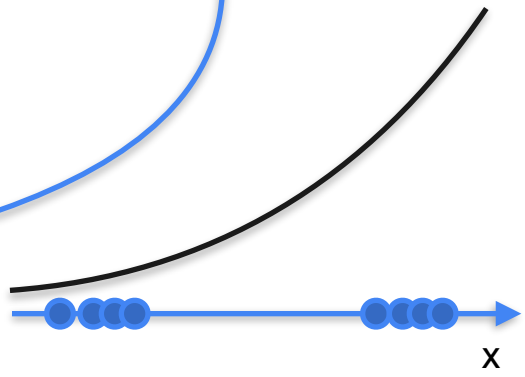
What is the difference, apart from discrete vs continuous?

$$\text{Risk}(f) = \mathbb{E}_{(x,y)}\left[\mathscr{L}(f(x), y)\right]$$

$$w^* = \arg\min_{f \in \mathscr{F}} \text{Risk}(f)$$

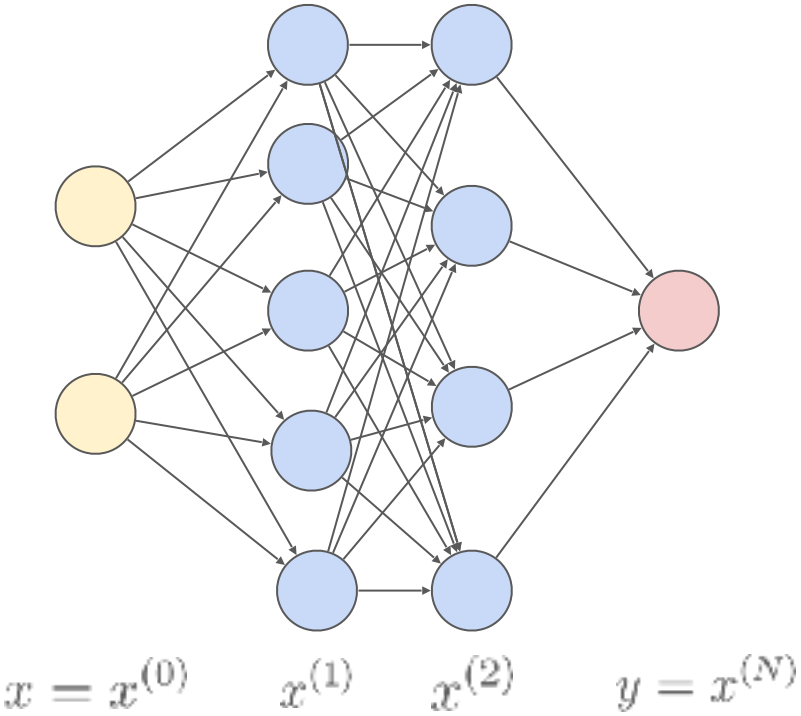
$$\mathscr{D} = \{(x^{(i)}, y^{(i)})\}_{i=0}^{K}, \quad x \in \mathbb{R}^n, y \in \mathbb{R}$$

$$\mathscr{L}'(w) = \frac{1}{N} \sum_{i=1}^{N} \left(y^{(i)} - f(x^{(i)}, w)\right)^2$$

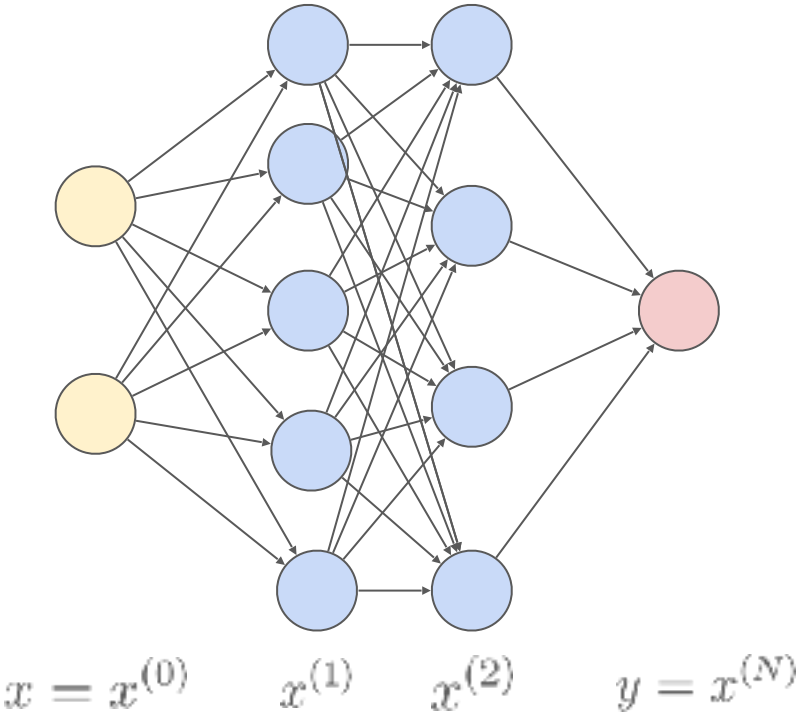


Classification:
the last layer and the loss function

How do we express y in classification?

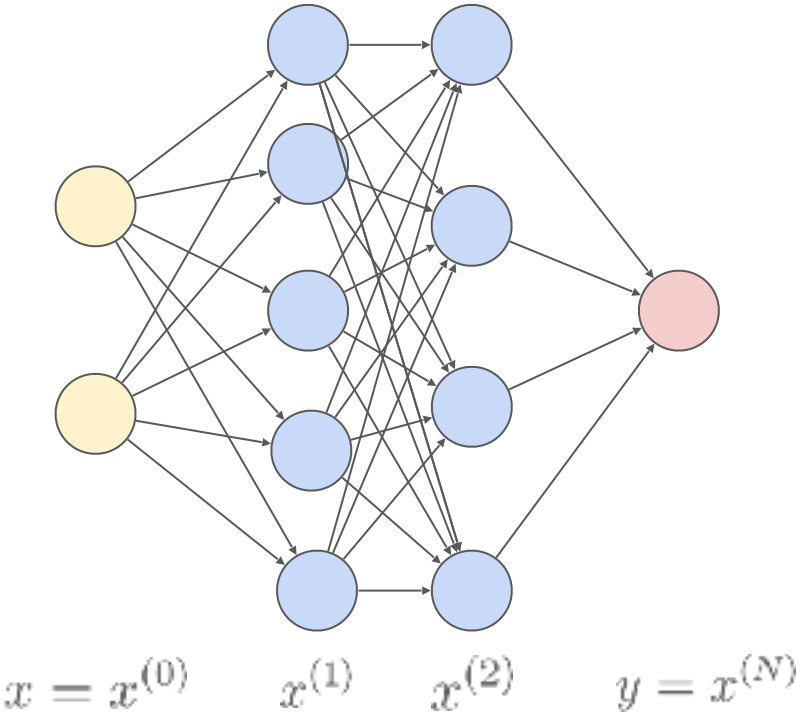


How do we express y in classification?



$$x^{(N)} = f(x, W) \approx y$$

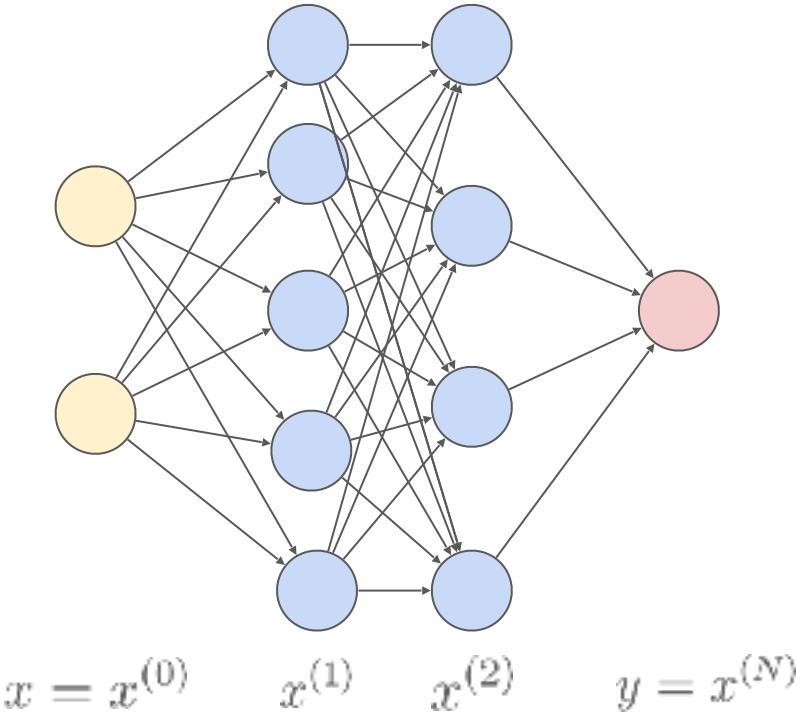
How do we express y in classification?



$$x^{(N)} = f(x, W) \approx y$$

$$y \in \{C_1, C_2, \ldots, C_K\}$$

How do we express y in classification?

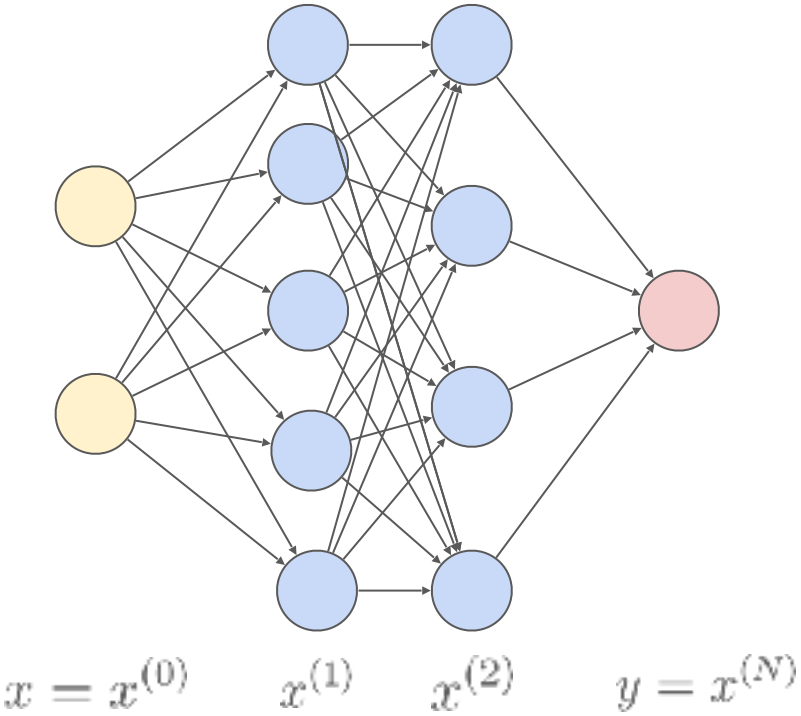


$$x^{(N)} = f(x, W) \approx y$$

$$y \in \{C_1, C_2, \ldots, C_K\}$$

How does output unit represent C_i?

How do we express y in classification?



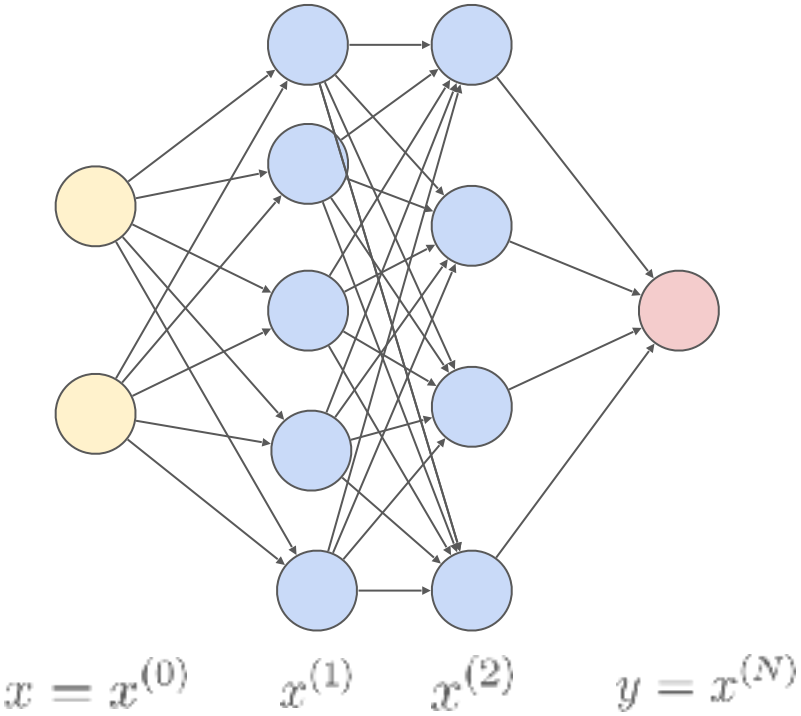
$$x^{(N)} = f(x, W) \approx y$$

$$y \in \{C_1, C_2, \ldots, C_K\}$$

How does output unit represent C_i?

E.g. map classes to natural numbers:

How do we express y in classification?



$$x^{(N)} = f(x, W) \approx y$$

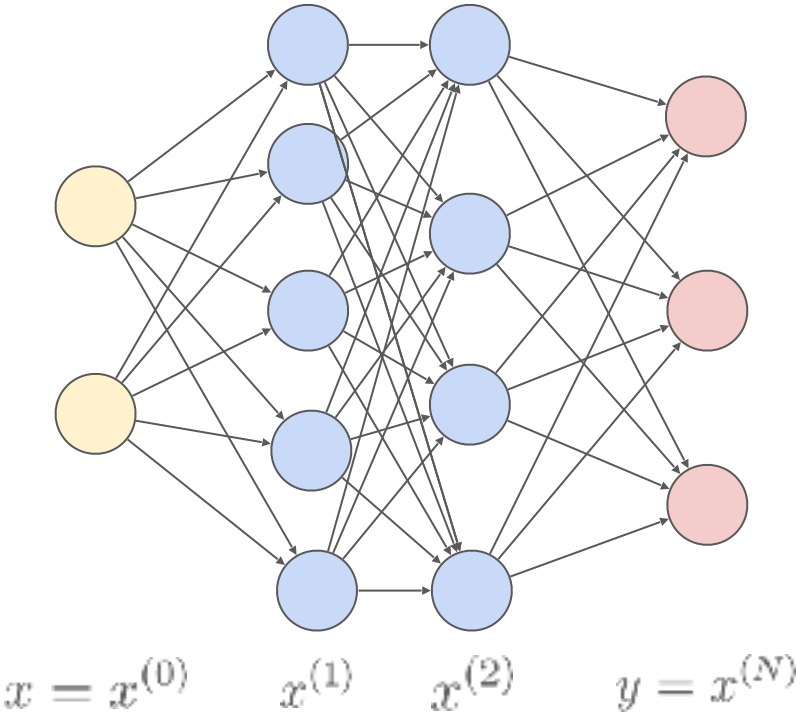
$$y \in \{C_1, C_2, \ldots, C_K\}$$

How does output unit represent C_i?

E.g. map classes to natural numbers:

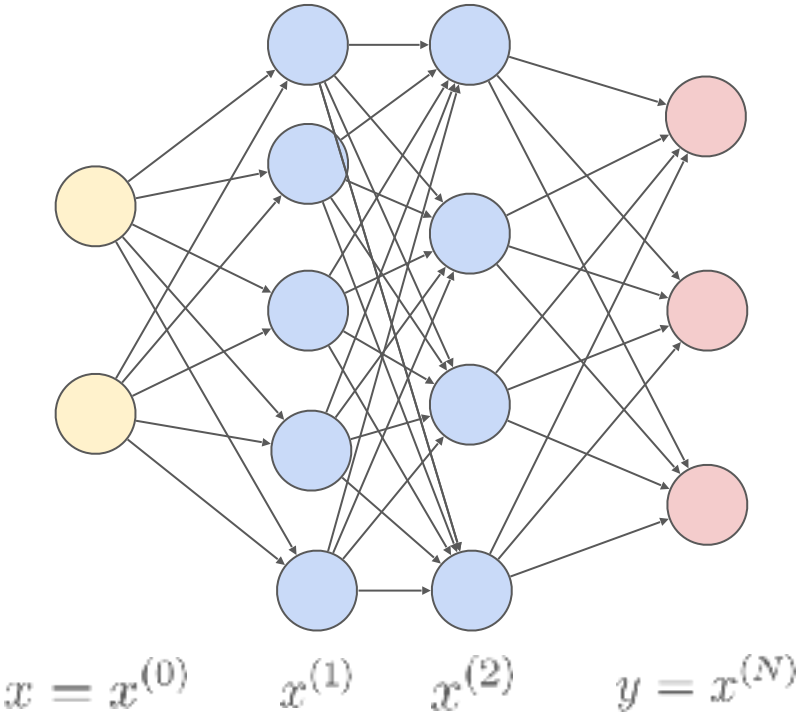
$$C_1 = 1,\ C_2 = 2,\ \ldots,\ C_K = K\ \ ?$$

Idea 1: 1-hot codes



Idea 1: 1-hot codes

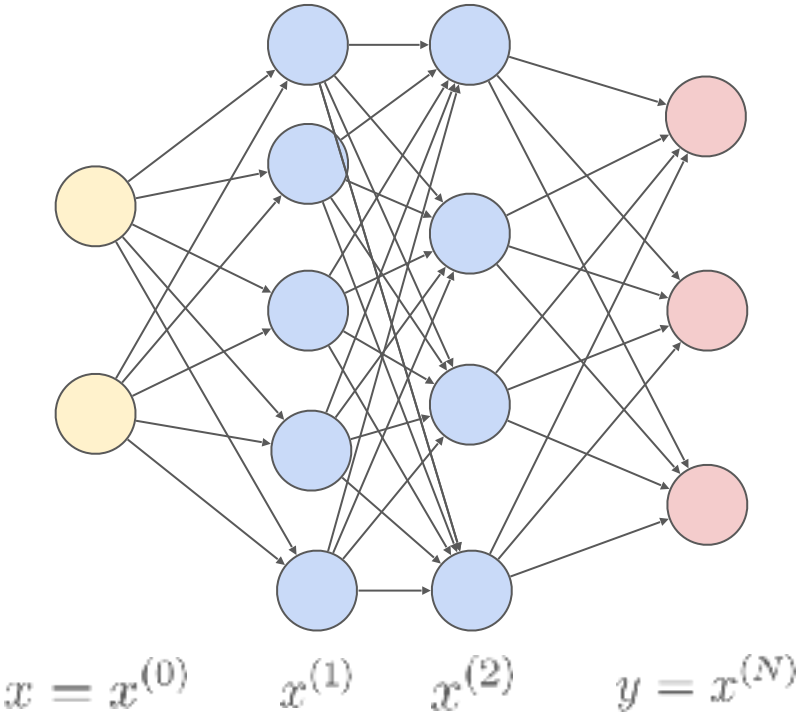
$$x^{(N)} = f(x, W) \approx y$$



Idea 1: 1-hot codes

$$x^{(N)} = f(x, W) \approx y$$

$$y \in \{C_1, C_2, \ldots, C_K\}$$

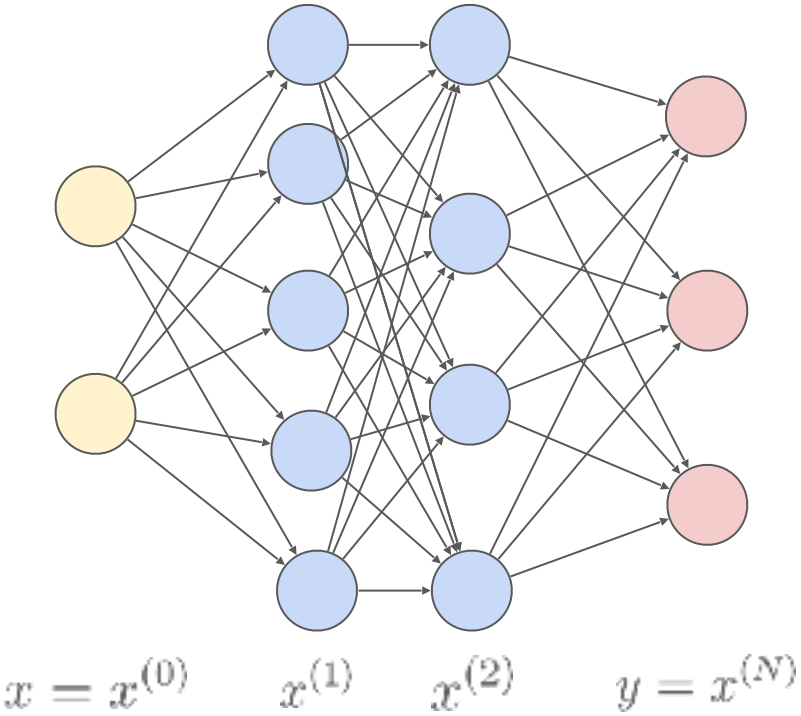


Idea 1: 1-hot codes

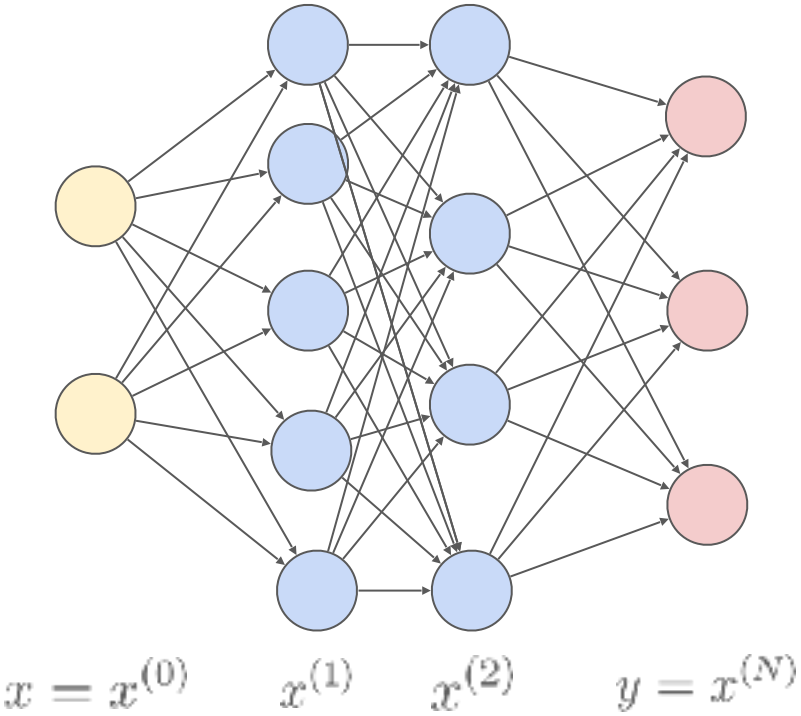
$$x^{(N)} = f(x, W) \approx y$$

$$y \in \{C_1, C_2, \ldots, C_K\}$$

Idea: 1-hot code:



Idea 1: 1-hot codes



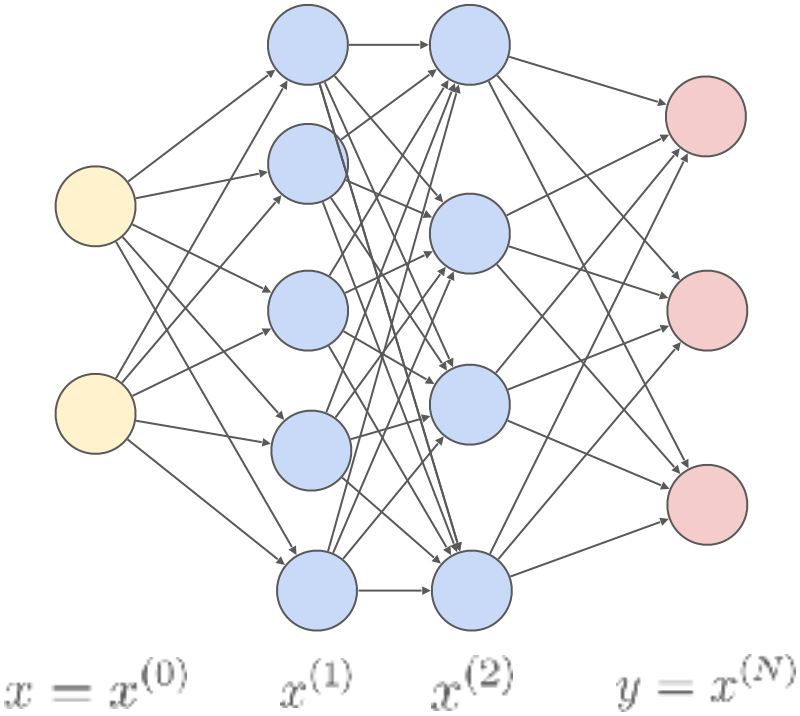
$$x^{(N)} = f(x, W) \approx y$$

$$y \in \{C_1, C_2, \ldots, C_K\}$$

Idea: 1-hot code:

$$C_1 = \begin{pmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}$$

Idea 1: 1-hot codes



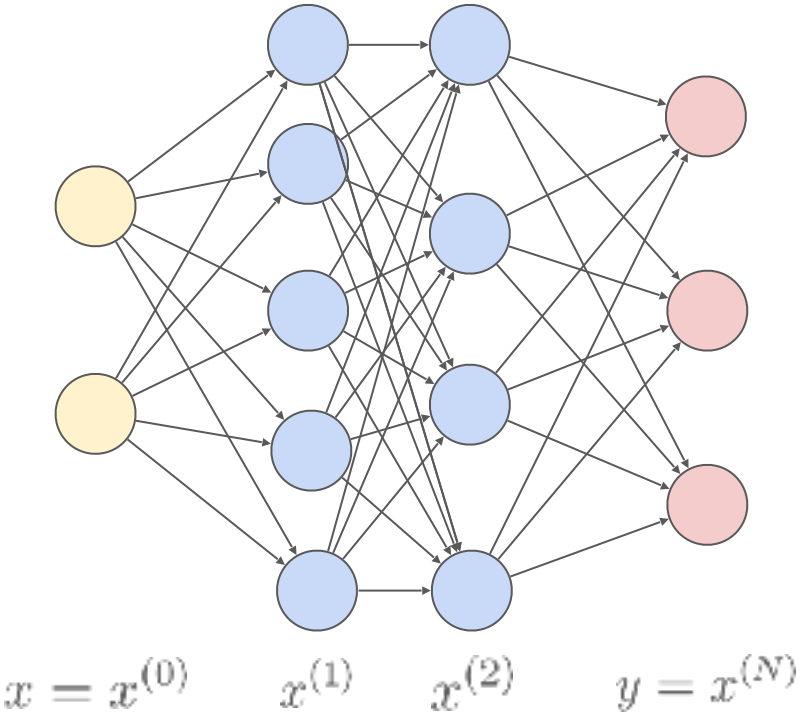
$$x^{(N)} = f(x, W) \approx y$$

$$y \in \{C_1, C_2, \ldots, C_K\}$$

Idea: 1-hot code:

$$C_1 = \begin{pmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix} \quad C_2 = \begin{pmatrix} 0 \\ 1 \\ \vdots \\ 0 \end{pmatrix}$$

Idea 1: 1-hot codes



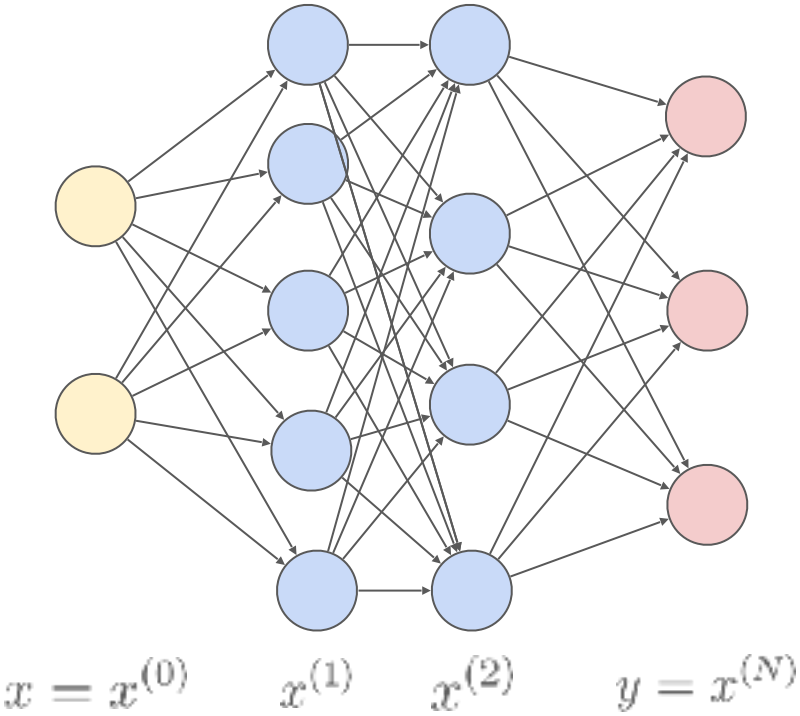
$$x^{(N)} = f(x, W) \approx y$$

$$y \in \{C_1, C_2, \ldots, C_K\}$$

Idea: 1-hot code:

$$C_1 = \begin{pmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix} \quad C_2 = \begin{pmatrix} 0 \\ 1 \\ \vdots \\ 0 \end{pmatrix} \qquad C_K = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 1 \end{pmatrix}$$

Idea 1: 1-hot codes



$$x^{(N)} = f(x, W) \approx y$$

$$y \in \{C_1, C_2, \ldots, C_K\}$$

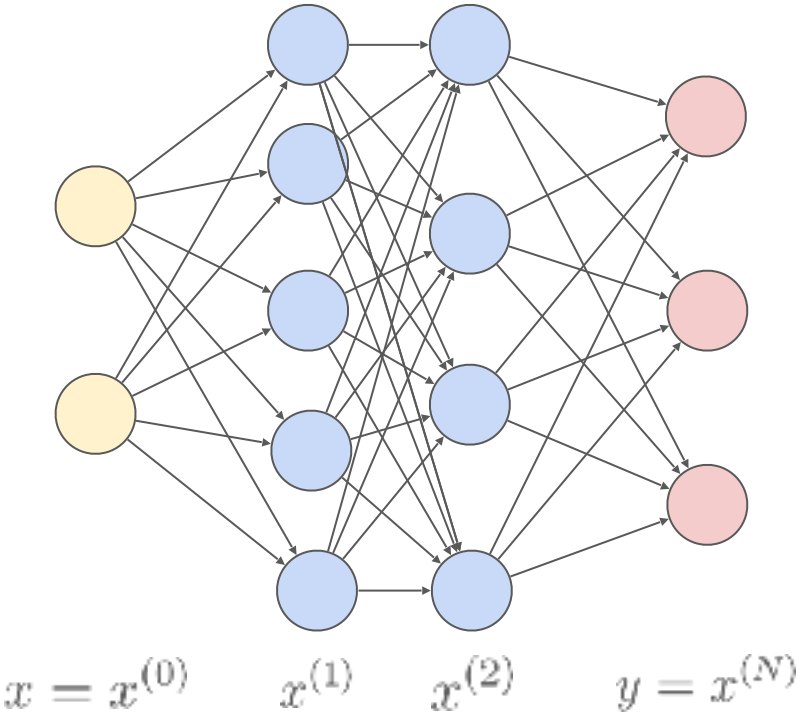
Idea: 1-hot code:

$$C_1 = \begin{pmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix} \quad C_2 = \begin{pmatrix} 0 \\ 1 \\ \vdots \\ 0 \end{pmatrix} \qquad C_K = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 1 \end{pmatrix}$$

Advantages:

- No order on set
- Can represent uncertainty between classes

Idea 1: 1-hot codes



$$x^{(N)} = f(x, W) \approx y$$

$$y \in \{C_1, C_2, \ldots, C_K\}$$

Idea: 1-hot code:

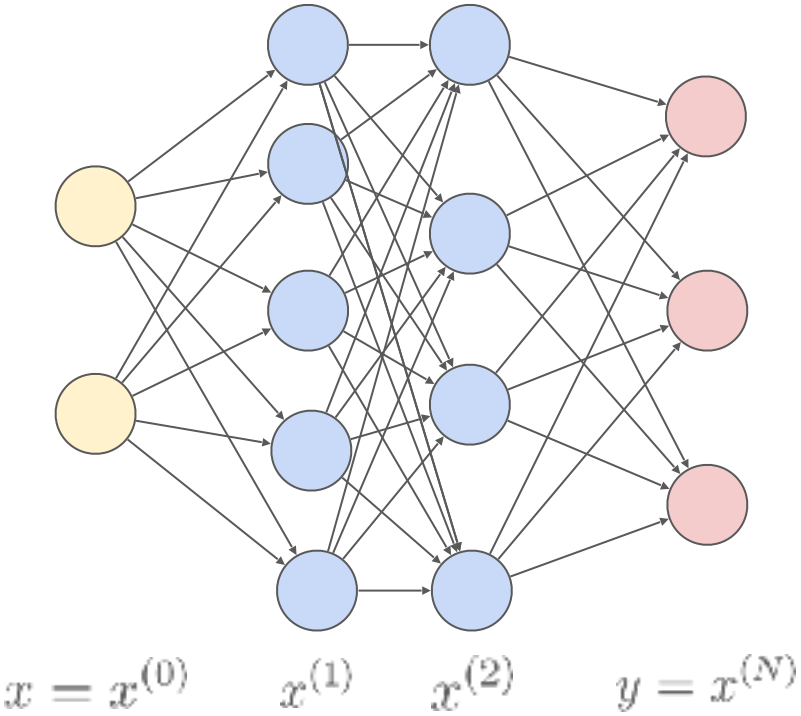
$$C_1 = \begin{pmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix} \quad C_2 = \begin{pmatrix} 0 \\ 1 \\ \vdots \\ 0 \end{pmatrix} \qquad C_K = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 1 \end{pmatrix}$$

Advantages:

- No order on set
- Can represent uncertainty between classes

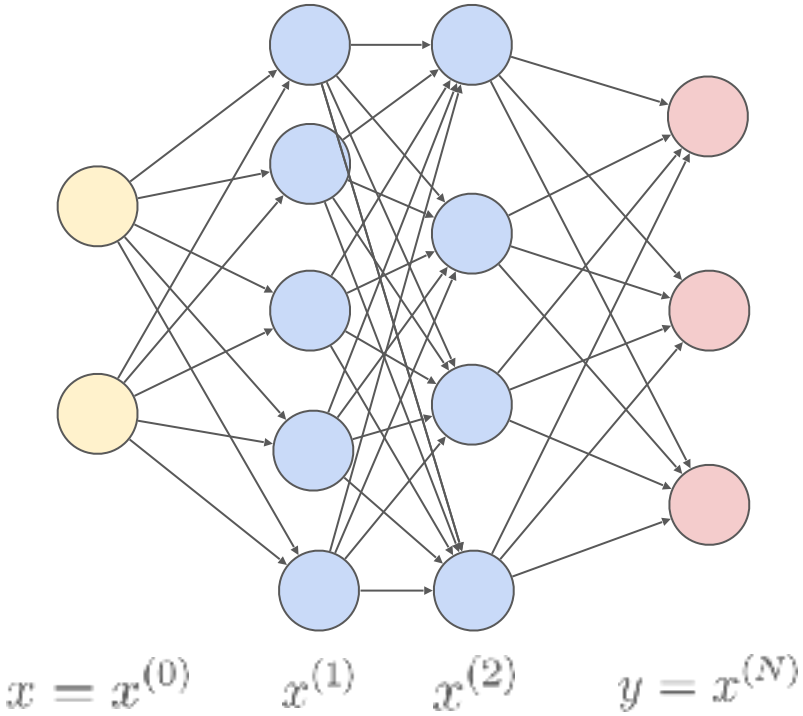
$$C_1 = \begin{pmatrix} 0.8 \\ 0.2 \\ \vdots \\ 0 \end{pmatrix}$$

Idea 2: soft-max



Idea 2: soft-max

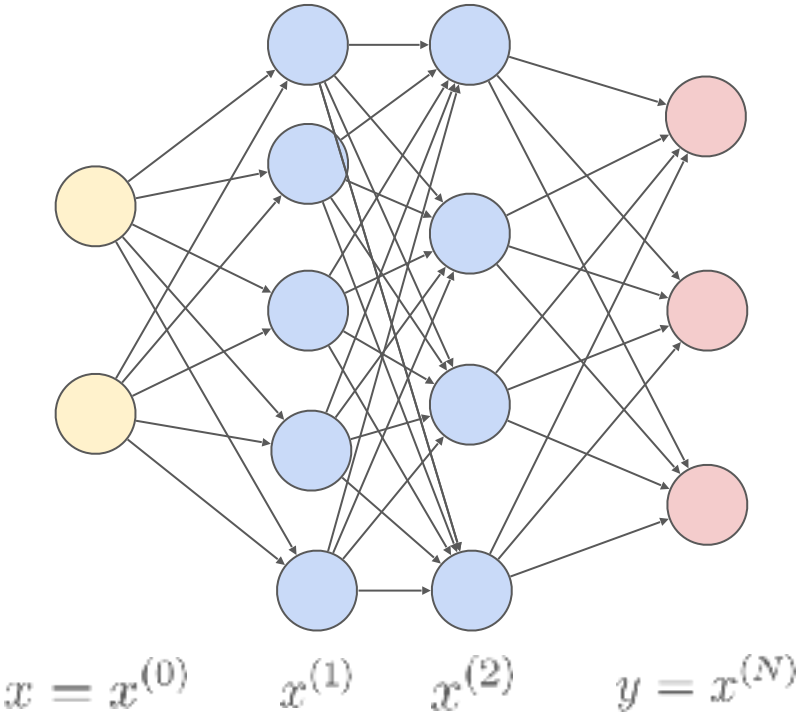
$$x^{(N)} = f(x, W) \approx y$$



Idea 2: soft-max

$$x^{(N)} = f(x, W) \approx y$$

$$y \in \{C_1, C_2, \ldots, C_K\}$$

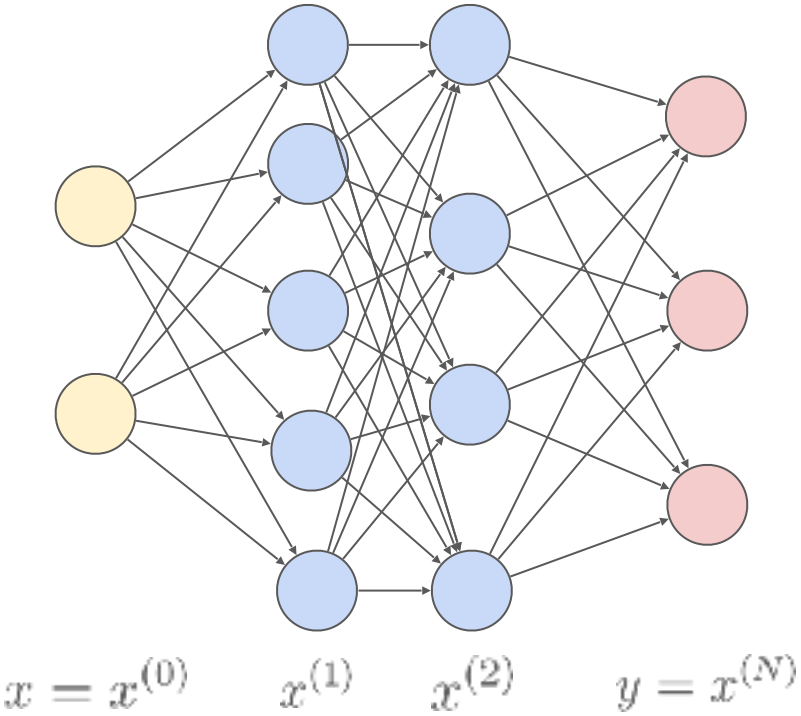


Idea 2: soft-max

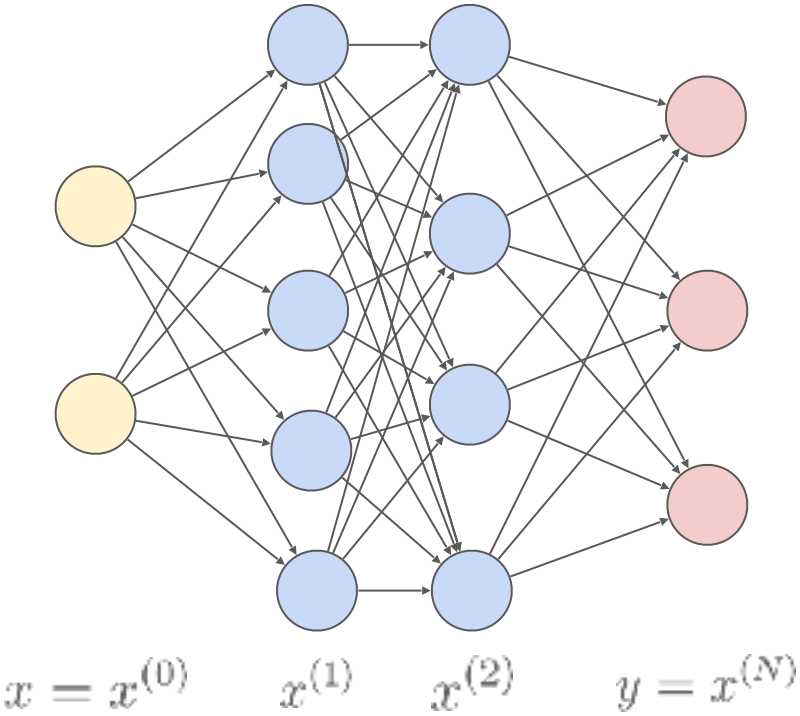
$$x^{(N)} = f(x, W) \approx y$$

$$y \in \{C_1, C_2, \ldots, C_K\}$$

How do we make y be prob-like?



Idea 2: soft-max



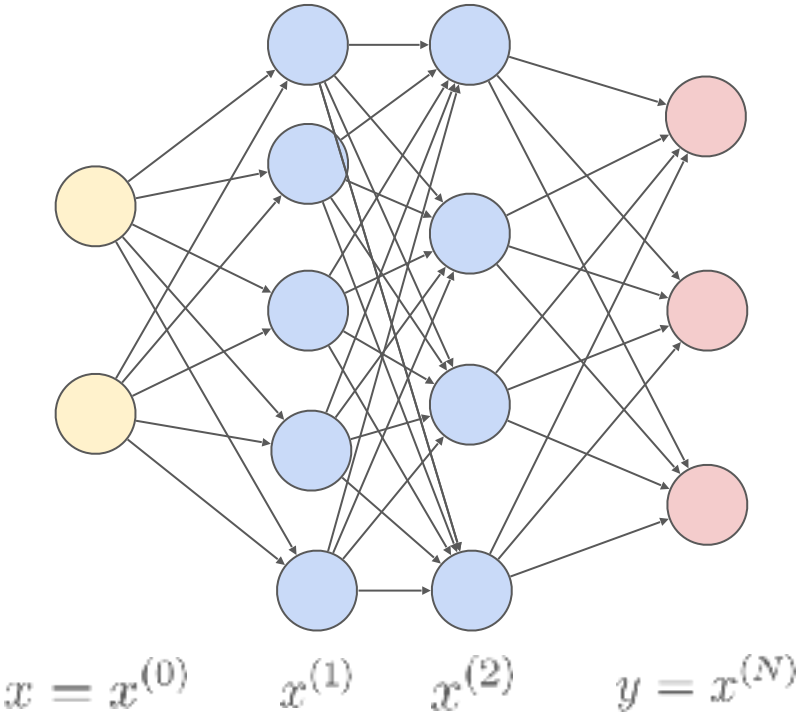
$$x^{(N)} = f(x, W) \approx y$$

$$y \in \{C_1, C_2, \ldots, C_K\}$$

How do we make y be prob-like?

$$x^{(N-1)} = \begin{pmatrix} 5 \\ 0.5 \\ \vdots \\ 0 \end{pmatrix} \longrightarrow x^{(N)} = \begin{pmatrix} 0.8 \\ 0.2 \\ \vdots \\ 0 \end{pmatrix}$$

Idea 2: soft-max



$$x^{(N)} = f(x, W) \approx y$$

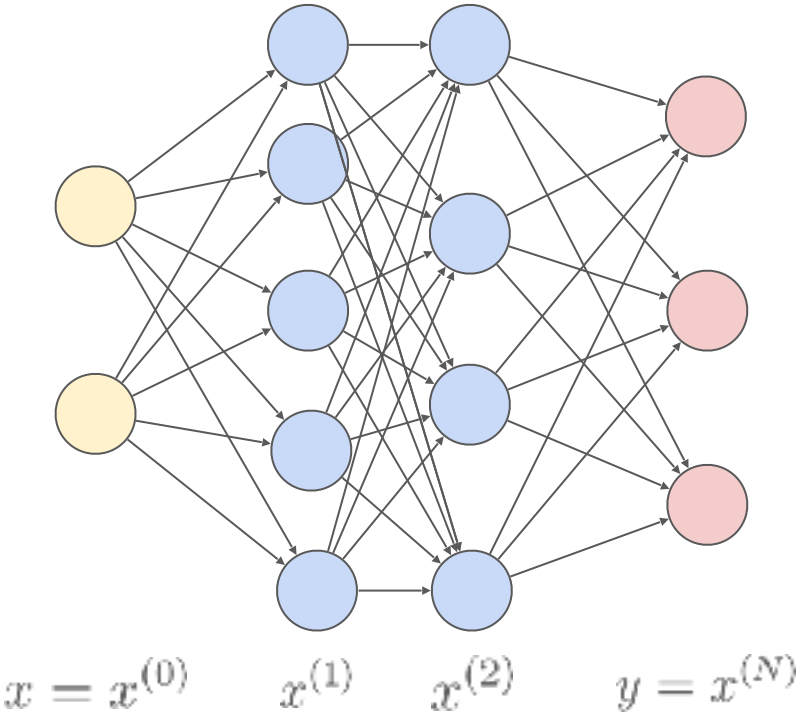
$$y \in \{C_1, C_2, \ldots, C_K\}$$

How do we make y be prob-like?

$$x^{(N-1)} = \begin{pmatrix} 5 \\ 0.5 \\ \vdots \\ 0 \end{pmatrix} \longrightarrow x^{(N)} = \begin{pmatrix} 0.8 \\ 0.2 \\ \vdots \\ 0 \end{pmatrix}$$

Softmax:

Idea 2: soft-max



$$x^{(N)} = f(x, W) \approx y$$

$$y \in \{C_1, C_2, \ldots, C_K\}$$

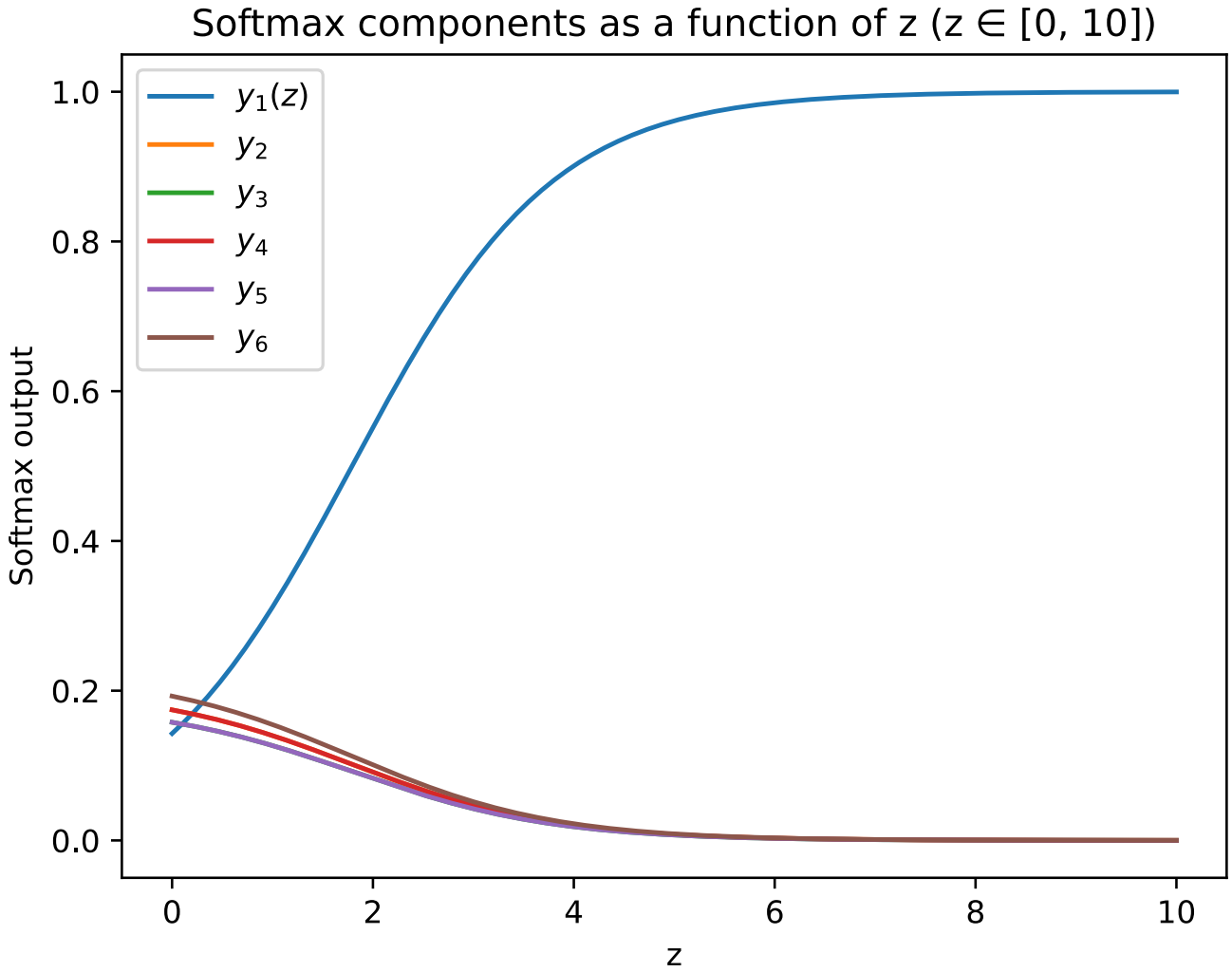
How do we make y be prob-like?

$$x^{(N-1)} = \begin{pmatrix} 5 \\ 0.5 \\ \vdots \\ 0 \end{pmatrix} \longrightarrow x^{(N)} = \begin{pmatrix} 0.8 \\ 0.2 \\ \vdots \\ 0 \end{pmatrix}$$

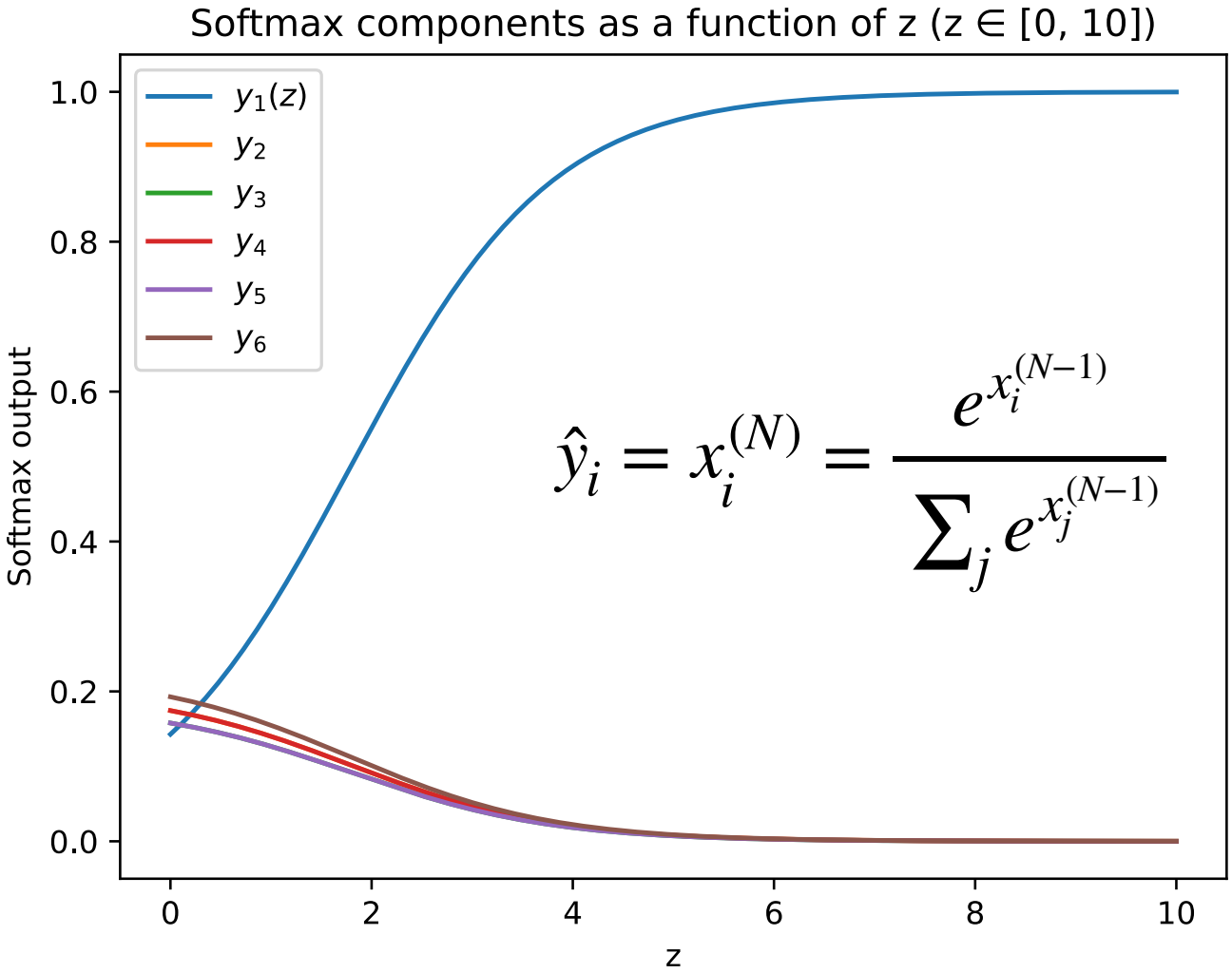
Softmax:

$$\hat{y}_i = x_i^{(N)} = \frac{e^{x_i^{(N-1)}}}{\sum_j e^{x_j^{(N-1)}}}$$

Softmax


Softmax components as a function of z (z ∈ [0, 10])
$y_1(z)$
y_2
y_3
y_4
y_5
y_6
Softmax output
1.0
0.8
0.6
0.4
0.2
0.0
0
2
4
6
8
10
z

Softmax



$$\hat{y}_i = x_i^{(N)} = \frac{e^{x_i^{(N-1)}}}{\sum_j e^{x_j^{(N-1)}}}$$

outputs ~ probs => cross-entropy loss

Soft-max to prob-like output:

Outputs ~ probabilities => $p_i = P(y = C_i \,|\, x)$

Likelihood of y given input: $P(y \,|\, x) = \prod_i p_i^{y_i}$

Negative log-likelihood: $\mathscr{L} = -\log P(y \mid x) = -\sum_i y_i \log p_i$

Cross-entropy loss: $\mathscr{L}_{CE}(y, p) = -\sum_i y_i \log p_i$

outputs ~ probs => cross-entropy loss

Soft-max to prob-like output: $$p_i = x_i^{(N)} = \frac{e^{x_i^{(N-1)}}}{\sum_j e^{x_j^{(N-1)}}}$$

Outputs ~ probabilities => $$p_i = P(y = C_i \,|\, x)$$

Likelihood of y given input: $$P(y \,|\, x) = \prod_i p_i^{y_i}$$

Negative log-likelihood: $$\mathscr{L} = -\log P(y \mid x) = -\sum_i y_i \log p_i$$

Cross-entropy loss: $$\mathscr{L}_{CE}(y, p) = -\sum_i y_i \log p_i$$

Why this is natural and elegant:

Soft-max to prob-like output:

Cross-entropy loss: $$\mathscr{L}_{CE}(y, p) = -\sum_i y_i \log p_i$$

Derivative of logic function: $$p_i = \mathrm{softmax}(z)_i = \frac{e^{z_i}}{\sum_k e^{z_k}} \longrightarrow \frac{\partial p_i}{\partial z_i} = p_i(1 - p_i)$$

Derivative of the loss w.r. to $x_i^{(N-1)}$: $$\frac{\partial \mathscr{L}}{\partial x_i^{(N-1)}} = p_i - y_i$$

Why this is natural and elegant:

Soft-max to prob-like output: $$p_i = x_i^{(N)} = \frac{e^{x_i^{(N-1)}}}{\sum_j e^{x_j^{(N-1)}}}$$

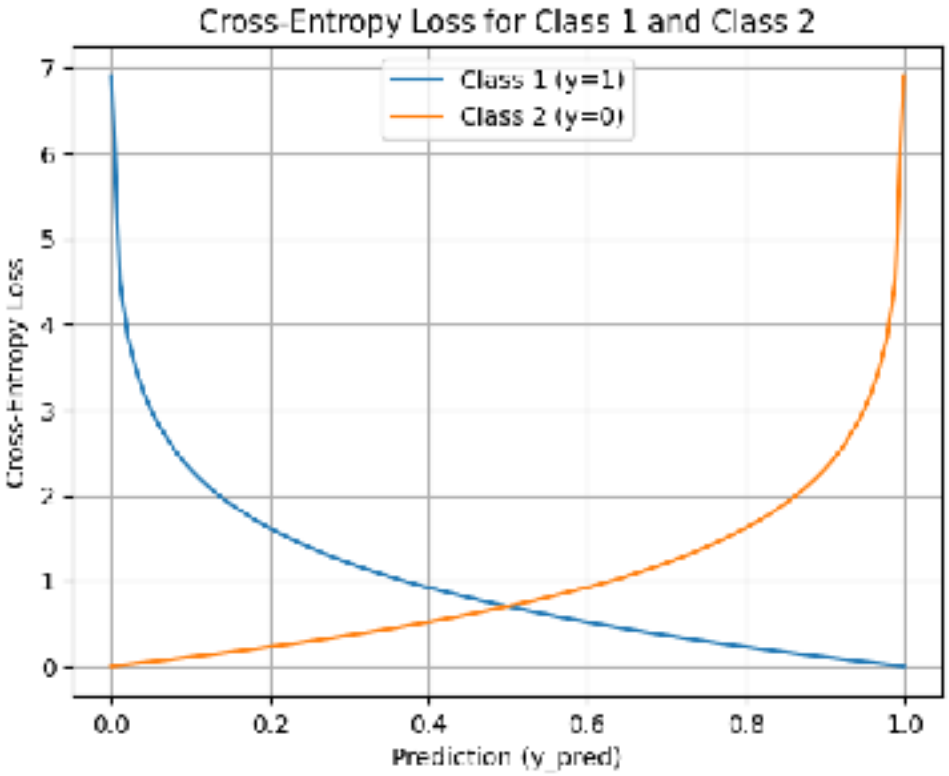
Cross-entropy loss: $$\mathscr{L}_{CE}(y, p) = -\sum_i y_i \log p_i$$

Derivative of logic function: $$p_i = \mathrm{softmax}(z)_i = \frac{e^{z_i}}{\sum_k e^{z_k}} \longrightarrow \frac{\partial p_i}{\partial z_i} = p_i(1 - p_i)$$

Derivative of the loss w.r. to $x_i^{(N-1)}$: $$\frac{\partial \mathscr{L}}{\partial x_i^{(N-1)}} = p_i - y_i$$

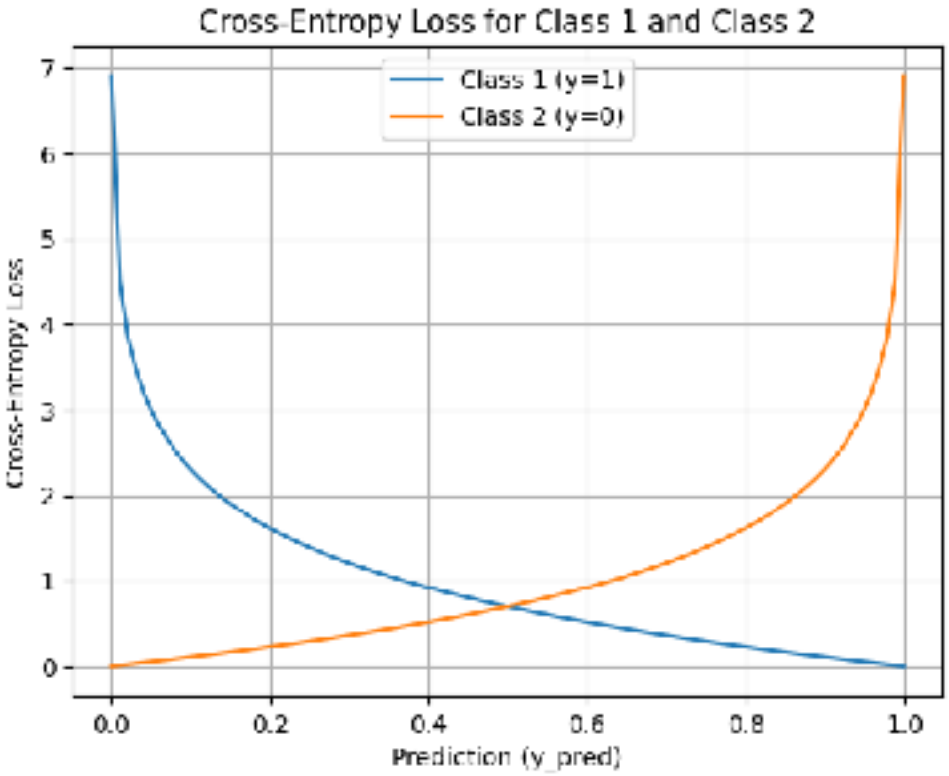
Cross entropy loss

$$\mathcal{L}(w) = -\frac{1}{K}\sum_{\mathcal{D}} \bar{y}^T log\, f(\bar{x}, w)$$



Cross entropy loss

$$\mathcal{L}(w) = -\frac{1}{K}\sum_{\mathcal{D}} \bar{y}^T log\, f(\bar{x}, w)$$



Other classification losses:

- Hinge loss
- Balanced Cross Entropy
- Focal Loss
- Contrastive loss
- …